

08_week8_knn_distance_metrics

July 30, 2026

Each week, you will apply the concepts of that week to your Integrated Capstone Project dataset. This notebook focuses on K-nearest neighbors, distance metrics, and how those choices behave differently on the Wisconsin and Brazil cancer datasets.

```
[1]: from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

PROJECT_ROOT
```

```
[1]: PosixPath('/Users/jamieconner/projects/bu/dx799_01/mod_c_capstone')
```

0.1 Imports

```
[2]: import matplotlib.pyplot as plt
import pandas as pd
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    average_precision_score,
    classification_report,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold, \
    train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from src.load_data import load_brazil, load_wisconsin, split_X_y
```

0.2 Setup

KNN is entirely distance-based, so scaling is required before comparing Euclidean, Manhattan, or Minkowski distance. I use stratified train/test splits and cross-validation to keep class balance stable and to reduce the chance that one lucky split drives the conclusion.

```
[3]: RANDOM_STATE = 42
BRAZIL_SAMPLE_SIZE = 10000
CV = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

BASE_PARAM_GRID = [
    {
        "knn__n_neighbors": [3, 5, 7, 11, 15, 21],
        "knn__weights": ["uniform", "distance"],
        "knn__metric": ["euclidean", "manhattan"],
    },
    {
        "knn__n_neighbors": [3, 5, 7, 11, 15, 21],
        "knn__weights": ["uniform", "distance"],
        "knn__metric": ["minkowski"],
        "knn__p": [1, 2],
    },
]

SCORING_OPTIONS = {
    "roc_auc": "roc_auc",
    "f1": "f1",
}

pd.set_option("display.max_columns", 100)
```

```
[4]: def run_knn_experiment(df, target_col, dataset_name, scoring="roc_auc"):
    X, y = split_X_y(df, target_col)
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        stratify=y,
        random_state=RANDOM_STATE,
    )

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier()),
    ])

    search = GridSearchCV(
        estimator=pipe,
```

```

        param_grid=BASE_PARAM_GRID,
        scoring=SCORING_OPTIONS[scoring],
        cv=CV,
        n_jobs=-1,
        refit=True,
    )
    search.fit(X_train, y_train)

    best_model = search.best_estimator_
    y_prob = best_model.predict_proba(X_test)[:, 1]
    y_pred = best_model.predict(X_test)

    results = {
        "dataset": dataset_name,
        "tuning_metric": scoring,
        "best_params": search.best_params_,
        "cv_score": search.best_score_,
        "test_accuracy": accuracy_score(y_test, y_pred),
        "test_precision": precision_score(y_test, y_pred, zero_division=0),
        "test_recall": recall_score(y_test, y_pred, zero_division=0),
        "test_f1": f1_score(y_test, y_pred, zero_division=0),
        "test_roc_auc": roc_auc_score(y_test, y_prob),
        "test_pr_auc": average_precision_score(y_test, y_prob),
    }

    cv_results = pd.DataFrame(search.cv_results_).sort_values(
        by="rank_test_score"
    )

    return {
        "results": results,
        "cv_results": cv_results,
        "best_model": best_model,
        "X_test": X_test,
        "y_test": y_test,
        "y_pred": y_pred,
        "y_prob": y_prob,
    }

def summarize_distance_results(cv_results, top_n=8):
    cols = [
        "mean_test_score",
        "std_test_score",
        "param_knn__metric",
        "param_knn__p",
        "param_knn__n_neighbors",
    ]

```

```

        "param_knn__weights",
        "rank_test_score",
    ]
    available_cols = [c for c in cols if c in cv_results.columns]
    return cv_results.loc[:, available_cols].head(top_n)

def plot_top_configs(cv_results, title, top_n=8):
    top = cv_results.head(top_n).copy()
    top["label"] = (
        top["param_knn__metric"].astype(str)
        + " | k="
        + top["param_knn__n_neighbors"].astype(str)
        + " | "
        + top["param_knn__weights"].astype(str)
    )

    plt.figure(figsize=(10, 4.5))
    plt.barh(top["label"], top["mean_test_score"], color="#2c7fb8")
    plt.gca().invert_yaxis()
    plt.xlabel("Mean CV score")
    plt.title(title)
    plt.tight_layout()
    plt.show()

def show_confusion(result, title):
    fig, ax = plt.subplots(figsize=(4.5, 4.5))
    ConfusionMatrixDisplay.from_predictions(
        result["y_test"],
        result["y_pred"],
        ax=ax,
        cmap="Blues",
        colorbar=False,
    )
    ax.set_title(title)
    plt.tight_layout()
    plt.show()

```

0.3 Wisconsin dataset

Wisconsin is the cleaner dataset, so it is a good place to check whether KNN can separate the classes after scaling and whether the choice of distance metric makes a meaningful difference.

```

[5]: df_w, target_w = load_wisconsin()
    wisconsin_roc = run_knn_experiment(df_w, target_w, "Wisconsin",
    ↪scoring="roc_auc")

```

```
wisconsin_f1 = run_knn_experiment(df_w, target_w, "Wisconsin", scoring="f1")

pd.DataFrame([wisconsin_roc["results"], wisconsin_f1["results"]])
```

```
[5]:      dataset tuning_metric                                best_params \
0  Wisconsin      roc_auc {'knn__metric': 'euclidean', 'knn__n_neighbors...
1  Wisconsin          f1 {'knn__metric': 'manhattan', 'knn__n_neighbors...

      cv_score test_accuracy test_precision test_recall test_f1 \
0  0.989216      0.947368      0.973684      0.880952      0.925
1  0.966708      0.964912      1.000000      0.904762      0.950

      test_roc_auc test_pr_auc
0      0.982308      0.979353
1      0.973545      0.965596
```

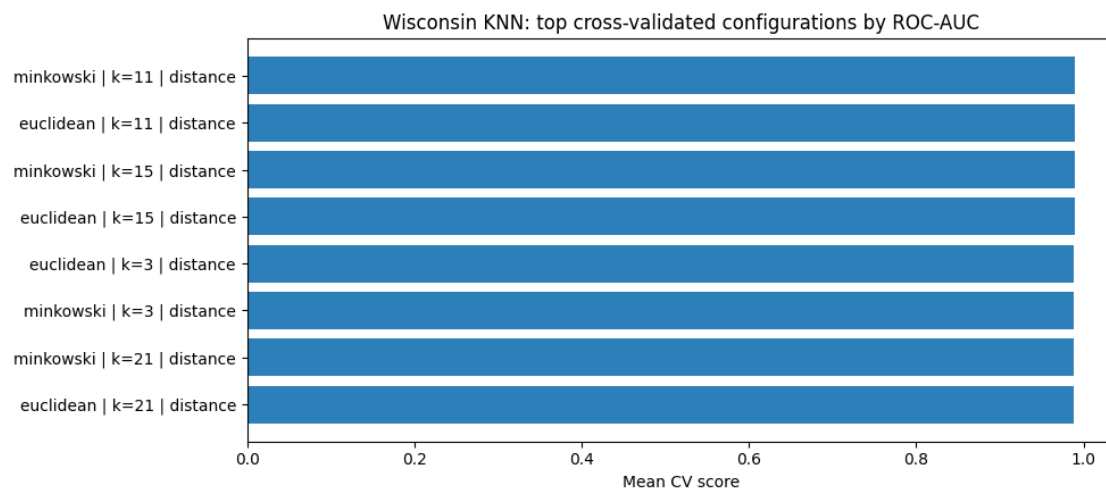
```
[6]: summarize_distance_results(wisconsin_roc["cv_results"])
```

```
[6]:      mean_test_score std_test_score param_knn__metric param_knn__p \
39      0.989216      0.010946      minkowski      2.0
7      0.989216      0.010946      euclidean      NaN
43      0.989112      0.009601      minkowski      2.0
9      0.989112      0.009601      euclidean      NaN
1      0.989009      0.012537      euclidean      NaN
27      0.989009      0.012537      minkowski      2.0
47      0.988854      0.009472      minkowski      2.0
11      0.988854      0.009472      euclidean      NaN

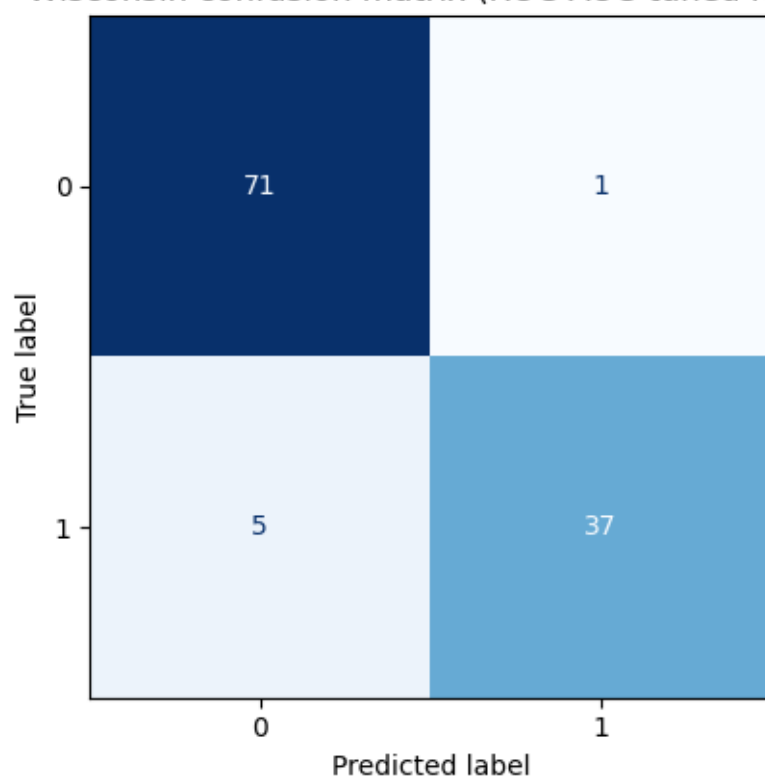
      param_knn__n_neighbors param_knn__weights rank_test_score
39      11      distance      1
7      11      distance      1
43      15      distance      3
9      15      distance      3
1      3      distance      5
27      3      distance      5
47      21      distance      7
11      21      distance      7
```

```
[7]: plot_top_configs(
      wisconsin_roc["cv_results"],
      "Wisconsin KNN: top cross-validated configurations by ROC-AUC",
)
show_confusion(wisconsin_roc, "Wisconsin confusion matrix (ROC-AUC-tuned KNN)")

print("Best ROC-AUC-tuned parameters:", wisconsin_roc["results"]["best_params"])
print(classification_report(wisconsin_roc["y_test"], wisconsin_roc["y_pred"],
↪zero_division=0))
```



Wisconsin confusion matrix (ROC-AUC-tuned KNN)



Best ROC-AUC-tuned parameters: {'knn__metric': 'euclidean', 'knn__n_neighbors': 11, 'knn__weights': 'distance'}

	precision	recall	f1-score	support
0	0.93	0.93	0.93	72
1	0.87	0.87	0.87	42

	0	0.93	0.99	0.96	72
	1	0.97	0.88	0.93	42
accuracy				0.95	114
macro avg		0.95	0.93	0.94	114
weighted avg		0.95	0.95	0.95	114

Wisconsin should perform strongly because the features are numeric, the dataset is relatively clean, and the classes are fairly separable. If Euclidean and Manhattan end up close, that supports the idea that the signal is strong enough that the metric choice matters less than basic scaling and a reasonable value of k .

0.4 Brazil dataset

Brazil is much larger and more imbalanced, so I use a stratified sample to keep KNN computationally feasible. This is still consistent with the milestone goal because the point is to compare how distance-based classification behaves on the harder dataset, especially in terms of recall and precision.

```
[8]: df_b, target_b = load_brazil(sample_size=BRAZIL_SAMPLE_SIZE,
    ↪random_state=RANDOM_STATE)
brazil_roc = run_knn_experiment(df_b, target_b, "Brazil", scoring="roc_auc")
brazil_f1 = run_knn_experiment(df_b, target_b, "Brazil", scoring="f1")

pd.DataFrame([brazil_roc["results"], brazil_f1["results"]])
```

```
[8]: dataset tuning_metric best_params \
0 Brazil      roc_auc {'knn__metric': 'euclidean', 'knn__n_neighbors...
1 Brazil      f1      {'knn__metric': 'euclidean', 'knn__n_neighbors...

      cv_score  test_accuracy  test_precision  test_recall  test_f1 \
0  0.909453      0.9385      0.648148      0.251799  0.362694
1  0.490359      0.9380      0.575758      0.410072  0.478992

      test_roc_auc  test_pr_auc
0      0.877595      0.440528
1      0.803933      0.384068
```

```
[9]: summarize_distance_results(brazil_roc["cv_results"])
```

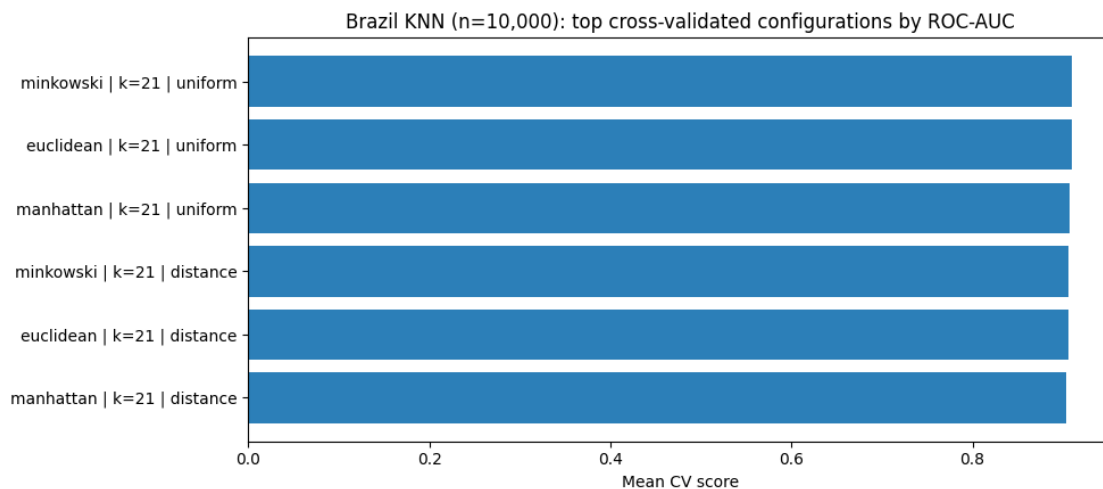
```
[9]: mean_test_score  std_test_score  param_knn__metric  param_knn__p \
46      0.909453      0.011346      minkowski      2.0
10      0.909453      0.011346      euclidean      NaN
44      0.907550      0.017167      minkowski      1.0
22      0.907550      0.017167      manhattan      NaN
47      0.905612      0.006689      minkowski      2.0
11      0.905612      0.006689      euclidean      NaN
```

45	0.903041	0.012209	minkowski	1.0
23	0.903041	0.012209	manhattan	NaN

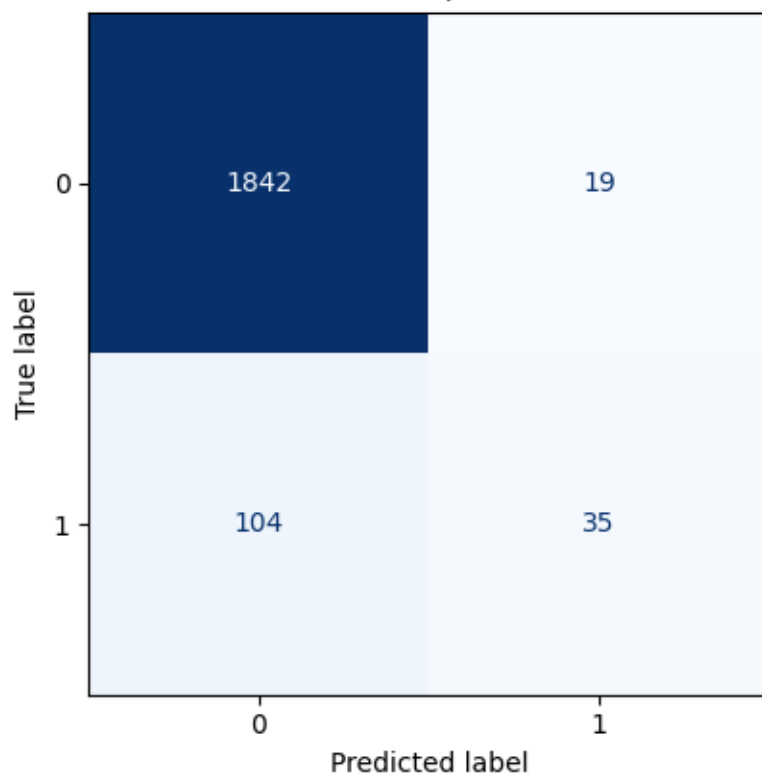
	param_knn__n_neighbors	param_knn__weights	rank_test_score
46	21	uniform	1
10	21	uniform	1
44	21	uniform	3
22	21	uniform	3
47	21	distance	5
11	21	distance	5
45	21	distance	7
23	21	distance	7

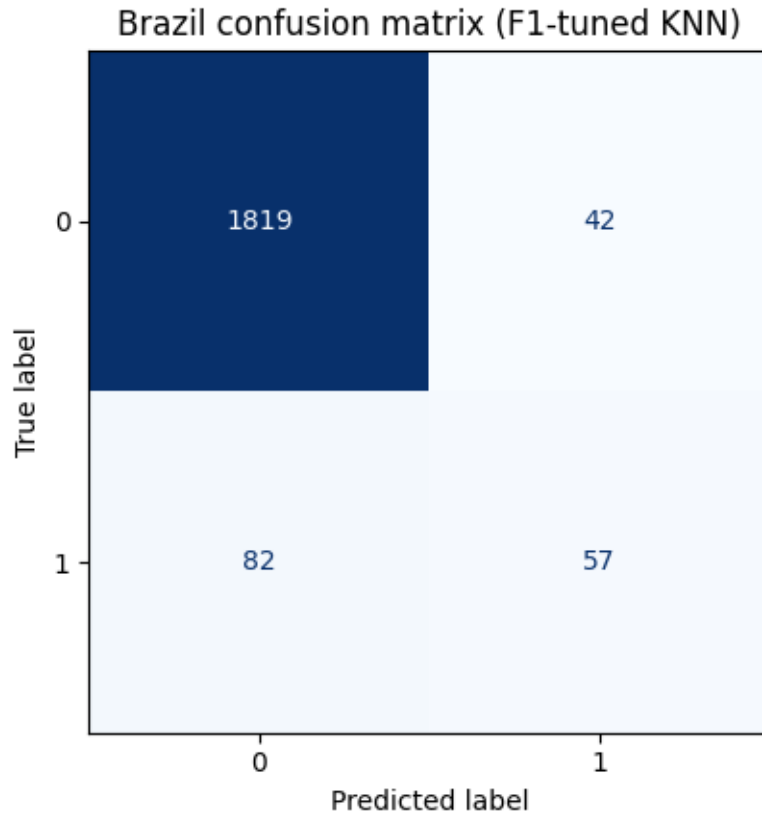
```
[10]: plot_top_configs(
    brazil_roc["cv_results"],
    f"Brazil KNN (n={BRAZIL_SAMPLE_SIZE:,}): top cross-validated configurations_
    ↳by ROC-AUC",
)
show_confusion(brazil_roc, "Brazil confusion matrix (ROC-AUC-tuned KNN)")
show_confusion(brazil_f1, "Brazil confusion matrix (F1-tuned KNN)")

print("Best ROC-AUC-tuned parameters:", brazil_roc["results"]["best_params"])
print(classification_report(brazil_roc["y_test"], brazil_roc["y_pred"],
    ↳zero_division=0))
print("Best F1-tuned parameters:", brazil_f1["results"]["best_params"])
print(classification_report(brazil_f1["y_test"], brazil_f1["y_pred"],
    ↳zero_division=0))
```



Brazil confusion matrix (ROC-AUC-tuned KNN)





Best ROC-AUC-tuned parameters: {'knn__metric': 'euclidean', 'knn__n_neighbors': 21, 'knn__weights': 'uniform'}

	precision	recall	f1-score	support
0	0.95	0.99	0.97	1861
1	0.65	0.25	0.36	139
accuracy			0.94	2000
macro avg	0.80	0.62	0.67	2000
weighted avg	0.93	0.94	0.93	2000

Best F1-tuned parameters: {'knn__metric': 'euclidean', 'knn__n_neighbors': 3, 'knn__weights': 'uniform'}

	precision	recall	f1-score	support
0	0.96	0.98	0.97	1861
1	0.58	0.41	0.48	139
accuracy			0.94	2000
macro avg	0.77	0.69	0.72	2000
weighted avg	0.93	0.94	0.93	2000

Brazil is the more informative KNN result for the capstone because accuracy alone can look high even when recall is still weak. A ROC-AUC-tuned model may rank cases well but miss too many positive cases at the default threshold, while an F1-tuned model usually trades some ranking quality for better balance between precision and recall.

0.5 Cross-dataset comparison

```
[11]: comparison = pd.DataFrame([
    wisconsin_roc["results"],
    wisconsin_f1["results"],
    brazil_roc["results"],
    brazil_f1["results"]
]).drop(columns=["best_params"])
comparison
```

```
[11]:
```

	dataset	tuning_metric	cv_score	test_accuracy	test_precision \
0	Wisconsin	roc_auc	0.989216	0.947368	0.973684
1	Wisconsin	f1	0.966708	0.964912	1.000000
2	Brazil	roc_auc	0.909453	0.938500	0.648148
3	Brazil	f1	0.490359	0.938000	0.575758

	test_recall	test_f1	test_roc_auc	test_pr_auc
0	0.880952	0.925000	0.982308	0.979353
1	0.904762	0.950000	0.973545	0.965596
2	0.251799	0.362694	0.877595	0.440528
3	0.410072	0.478992	0.803933	0.384068

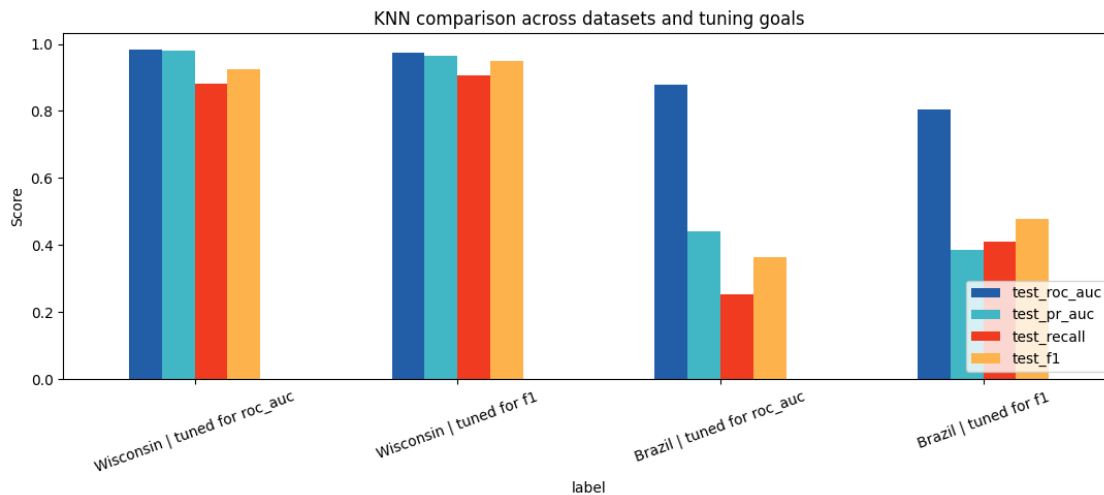
```
[12]: plot_df = comparison[[
    "dataset",
    "tuning_metric",
    "test_roc_auc",
    "test_pr_auc",
    "test_recall",
    "test_f1"
]].copy()
plot_df["label"] = plot_df["dataset"] + " | tuned for " + \
    plot_df["tuning_metric"]
plot_df = plot_df.set_index("label")

ax = plot_df[["test_roc_auc", "test_pr_auc", "test_recall", "test_f1"]].plot(
    kind="bar",
    figsize=(11, 5),
    rot=20,
    color=["#225ea8", "#41b6c4", "#f03b20", "#feb24c"]
)
ax.set_ylabel("Score")
```

```

ax.set_title("KNN comparison across datasets and tuning goals")
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()

```



```

[13]: # Report both ROC-AUC and PR-AUC CV performance for the selected KNN models.
# This refits only the winning pipelines; it does not repeat either grid search.
from sklearn.model_selection import cross_validate

knn_cv_scoring = {
    "roc_auc": "roc_auc",
    "average_precision": "average_precision",
}

X_w_all, y_w_all = split_X_y(df_w, target_w)
X_w_train, _, y_w_train, _ = train_test_split(
    X_w_all,
    y_w_all,
    test_size=0.2,
    stratify=y_w_all,
    random_state=RANDOM_STATE,
)

X_b_all, y_b_all = split_X_y(df_b, target_b)
X_b_train, _, y_b_train, _ = train_test_split(
    X_b_all,
    y_b_all,
    test_size=0.2,
    stratify=y_b_all,
    random_state=RANDOM_STATE,
)

```

```

wisconsin_knn_cv = cross_validate(
    wisconsin_roc["best_model"],
    X_w_train,
    y_w_train,
    cv=CV,
    scoring=knn_cv_scoring,
    n_jobs=1,
)
brazil_knn_cv = cross_validate(
    brazil_roc["best_model"],
    X_b_train,
    y_b_train,
    cv=CV,
    scoring=knn_cv_scoring,
    n_jobs=1,
)

knn_cv_results_df = pd.DataFrame(
    [
        {
            "dataset": "Wisconsin",
            "model": "KNN",
            "cv_folds": 5,
            "cv_roc_auc_mean": wisconsin_knn_cv["test_roc_auc"].mean(),
            "cv_roc_auc_sd": wisconsin_knn_cv["test_roc_auc"].std(),
            "cv_pr_auc_mean": wisconsin_knn_cv["test_average_precision"].mean(),
            "cv_pr_auc_sd": wisconsin_knn_cv["test_average_precision"].std(),
        },
        {
            "dataset": "Brazil",
            "model": "KNN",
            "cv_folds": 5,
            "cv_roc_auc_mean": brazil_knn_cv["test_roc_auc"].mean(),
            "cv_roc_auc_sd": brazil_knn_cv["test_roc_auc"].std(),
            "cv_pr_auc_mean": brazil_knn_cv["test_average_precision"].mean(),
            "cv_pr_auc_sd": brazil_knn_cv["test_average_precision"].std(),
        },
    ]
)

print(knn_cv_results_df.round(4).to_string(index=False))

```

dataset	model	cv_folds	cv_roc_auc_mean	cv_roc_auc_sd	cv_pr_auc_mean	cv_pr_auc_sd
Wisconsin	KNN	5	0.9892	0.0109	0.9875	0.0121
Brazil	KNN	5	0.9094	0.0113	0.5165	

0.0429

0.6 Conclusions

KNN worked well on Wisconsin after scaling, which is what I expected from a cleaner and more separable dataset. Brazil was harder: the model could still produce useful ranking performance, but the default classification behavior showed why recall, F1, and PR-AUC matter more than accuracy alone on an imbalanced outcome problem.

For the Milestone 2 writeup, the most defensible KNN discussion is:

- Overfitting was controlled with scaling, train/test separation, stratified cross-validation, and tuning k instead of using a very small neighborhood by default.
- Distance metric choice mattered somewhat, but the larger lesson was that dataset structure mattered more: Wisconsin was easy for KNN, while Brazil exposed the precision-recall tradeoff.
- EDA mattered because KNN depends directly on distance, so feature scale and class imbalance were both central to model behavior.

09_week9_gradient_boosting

July 30, 2026

1 Week 9: Gradient Boosting on My Two Datasets

For this week, I wanted to try gradient boosting on both of the supervised datasets I have been using in this project:

1. Wisconsin breast cancer diagnosis
2. Brazil patient outcome dataset

The main thing I wanted to see was how gradient boosting compares to random forest on each one. I also wanted to keep this notebook pretty straightforward, so I am doing the model fitting and graphs directly in the notebook instead of hiding a lot of the work in helper functions.

```
[17]: from pathlib import Path
import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier, \
    GradientBoostingClassifier, HistGradientBoostingClassifier
from sklearn.inspection import PartialDependenceDisplay, permutation_importance
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold, \
    train_test_split

project_root = Path.cwd().resolve()
if not (project_root / "src").exists():
```

```

    project_root = project_root.parent
    if str(project_root) not in sys.path:
        sys.path.append(str(project_root))

    from src.load_data import load_wisconsin, load_brazil, split_X_y

    sns.set_theme(style="whitegrid", context="notebook")
    pd.set_option("display.max_columns", 100)

```

1.1 1. Wisconsin Dataset

I started with Wisconsin because it is smaller and cleaner, which makes it a good dataset for testing the full gradient boosting workflow.

In this section, I am checking the class balance, splitting the data into training and test sets, tuning a few gradient boosting settings, and then comparing the results to random forest.

```

[18]: wisconsin_df, wisconsin_target = load_wisconsin()
      X_w, y_w = split_X_y(wisconsin_df, wisconsin_target)

      print("Wisconsin shape:", wisconsin_df.shape)
      display(y_w.value_counts().rename("count").to_frame())
      display((y_w.value_counts(normalize=True).rename("share") * 100).round(2).
               ↪to_frame())

      X_w_train, X_w_test, y_w_train, y_w_test = train_test_split(
          X_w, y_w, test_size=0.25, random_state=42, stratify=y_w
      )

      print("Training rows:", X_w_train.shape[0])
      print("Test rows:", X_w_test.shape[0])

```

Wisconsin shape: (569, 31)

	count
Diagnosis	
0	357
1	212

	share
Diagnosis	
0	62.74
1	37.26

Training rows: 426

Test rows: 143

```

[19]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

      wisconsin_rf = RandomForestClassifier(

```

```

        n_estimators=300,
        min_samples_leaf=2,
        random_state=42,
        n_jobs=-1,
    )
    wisconsin_rf.fit(X_w_train, y_w_train)

    wisconsin_gb_grid = {
        "n_estimators": [100, 200, 300],
        "learning_rate": [0.03, 0.05, 0.1],
        "max_depth": [1, 2, 3],
        "subsample": [0.8, 1.0],
    }

    wisconsin_gb_search = GridSearchCV(
        estimator=GradientBoostingClassifier(random_state=42),
        param_grid=wisconsin_gb_grid,
        scoring="roc_auc",
        cv=cv,
        n_jobs=-1,
    )
    wisconsin_gb_search.fit(X_w_train, y_w_train)

    wisconsin_gb = wisconsin_gb_search.best_estimator_
    print("Best Wisconsin gradient boosting parameters:")
    print(wisconsin_gb_search.best_params_)

```

Best Wisconsin gradient boosting parameters:

```
{'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 300, 'subsample': 0.8}
```

```

[20]: wisconsin_results = []

for model_name, model in {
    "Random Forest": wisconsin_rf,
    "Gradient Boosting": wisconsin_gb,
}.items():
    y_pred = model.predict(X_w_test)
    y_score = model.predict_proba(X_w_test)[: , 1]

    wisconsin_results.append(
        {
            "model": model_name,
            "accuracy": accuracy_score(y_w_test, y_pred),
            "balanced_accuracy": balanced_accuracy_score(y_w_test, y_pred),
            "f1": f1_score(y_w_test, y_pred),
            "roc_auc": roc_auc_score(y_w_test, y_score),
            "average_precision": average_precision_score(y_w_test, y_score),
        }
    )

```

```

    }
)

wisconsin_results_df = pd.DataFrame(wisconsin_results).sort_values("roc_auc",
↪ascending=False)
display(wisconsin_results_df.round(4))

```

	model	accuracy	balanced_accuracy	f1	roc_auc \
1	Gradient Boosting	0.965	0.9528	0.9505	0.9977
0	Random Forest	0.965	0.9528	0.9505	0.9958

	average_precision
1	0.9966
0	0.9935

```

[21]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for model_name, model, color in [
    ("Random Forest", wisconsin_rf, "#3b82f6"),
    ("Gradient Boosting", wisconsin_gb, "#ef4444"),
]:
    y_pred = model.predict(X_w_test)
    y_score = model.predict_proba(X_w_test)[:, 1]

    fpr, tpr, _ = roc_curve(y_w_test, y_score)
    precision, recall, _ = precision_recall_curve(y_w_test, y_score)

    axes[0].plot(fpr, tpr, label=model_name, color=color, linewidth=2)
    axes[1].plot(recall, precision, label=model_name, color=color, linewidth=2)

cm = confusion_matrix(y_w_test, wisconsin_gb.predict(X_w_test))
sns.heatmap(cm, annot=True, fmt="d", cmap="Reds", cbar=False, ax=axes[2])

axes[0].plot([0, 1], [0, 1], linestyle="--", color="gray")
axes[0].set_title("Wisconsin ROC Curve")
axes[0].set_xlabel("False Positive Rate")
axes[0].set_ylabel("True Positive Rate")
axes[0].legend()

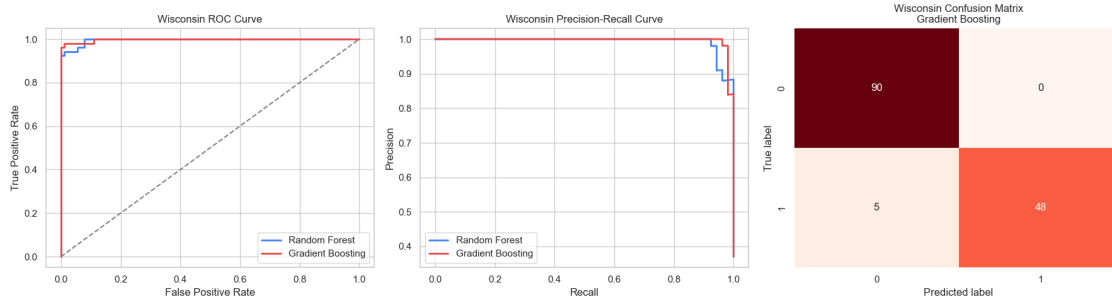
axes[1].set_title("Wisconsin Precision-Recall Curve")
axes[1].set_xlabel("Recall")
axes[1].set_ylabel("Precision")
axes[1].legend()

axes[2].set_title("Wisconsin Confusion Matrix\nGradient Boosting")
axes[2].set_xlabel("Predicted label")
axes[2].set_ylabel("True label")

```

```
plt.tight_layout()
plt.show()

print("Wisconsin classification report for gradient boosting:")
print(classification_report(y_w_test, wisconsin_gb.predict(X_w_test)))
```

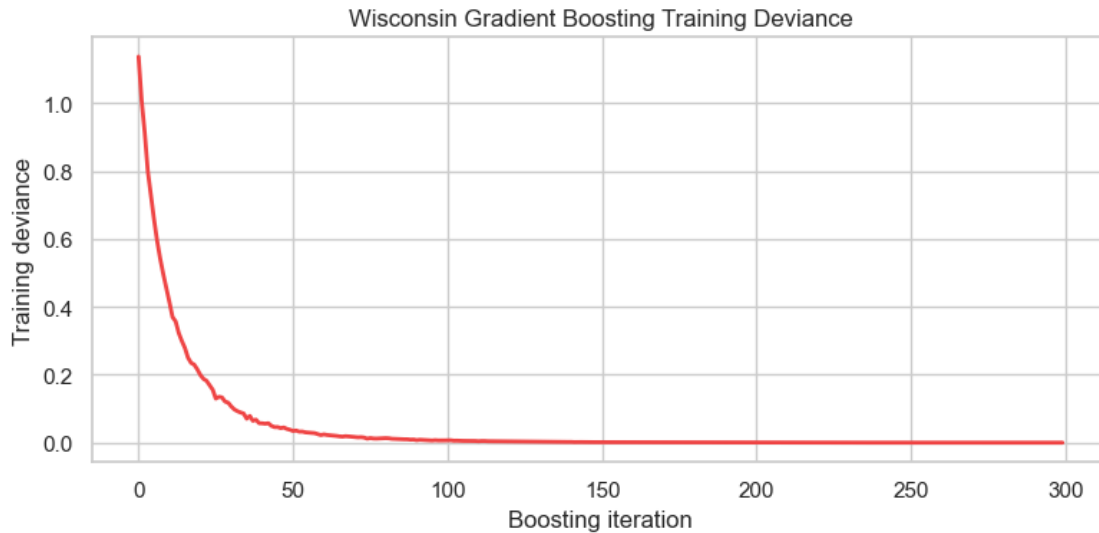


Wisconsin classification report for gradient boosting:

	precision	recall	f1-score	support
0	0.95	1.00	0.97	90
1	1.00	0.91	0.95	53
accuracy			0.97	143
macro avg	0.97	0.95	0.96	143
weighted avg	0.97	0.97	0.96	143

```
[22]: fig, ax = plt.subplots(figsize=(8, 4))

ax.plot(wisconsin_gb.train_score_, color="#ef4444", linewidth=2)
ax.set_title("Wisconsin Gradient Boosting Training Deviance")
ax.set_xlabel("Boosting iteration")
ax.set_ylabel("Training deviance")
plt.tight_layout()
plt.show()
```



```
[38]: wisconsin_perm = permutation_importance(
    wisconsin_gb,
    X_w_test,
    y_w_test,
    n_repeats=20,
    random_state=42,
    scoring="roc_auc",
)

wisconsin_importance = (
    pd.DataFrame(
        {
            "feature": X_w.columns,
            "importance": wisconsin_perm.importances_mean,
        }
    )
    .sort_values("importance", ascending=False)
    .head(10)
)

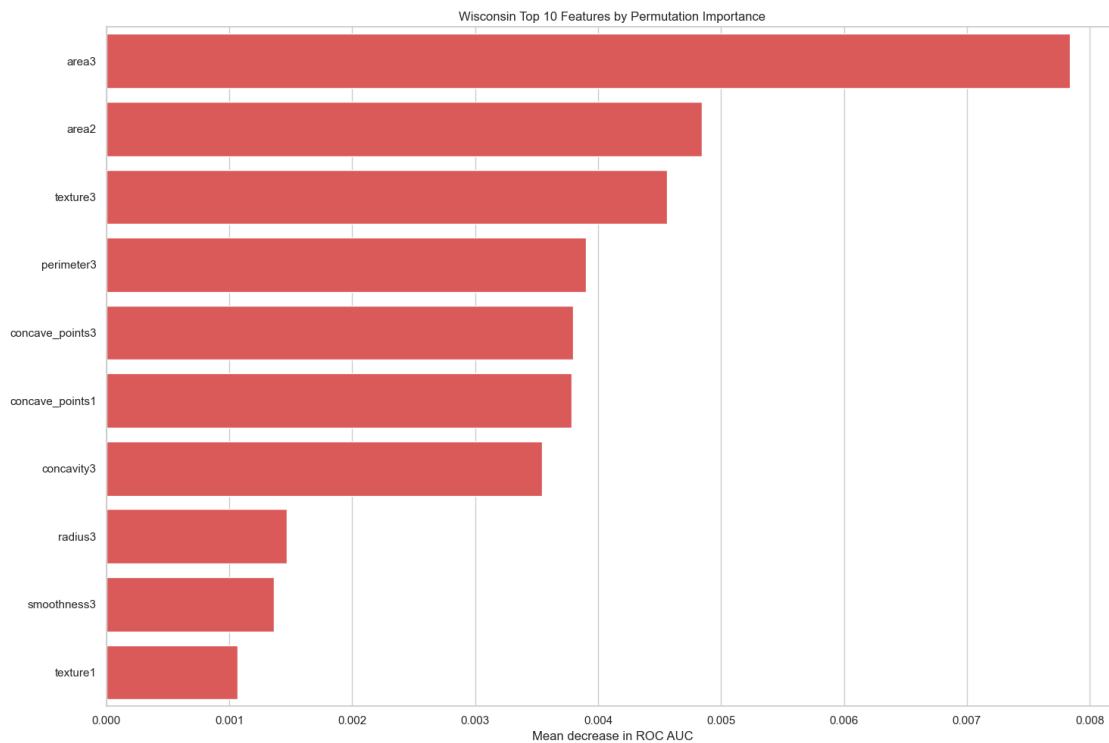
plt.figure(figsize=(15, 10))
sns.barplot(data=wisconsin_importance, x="importance", y="feature",
            color="#ef4444")
plt.title("Wisconsin Top 10 Features by Permutation Importance")
plt.xlabel("Mean decrease in ROC AUC")
plt.ylabel("")
plt.tight_layout()
plt.show()
```

```

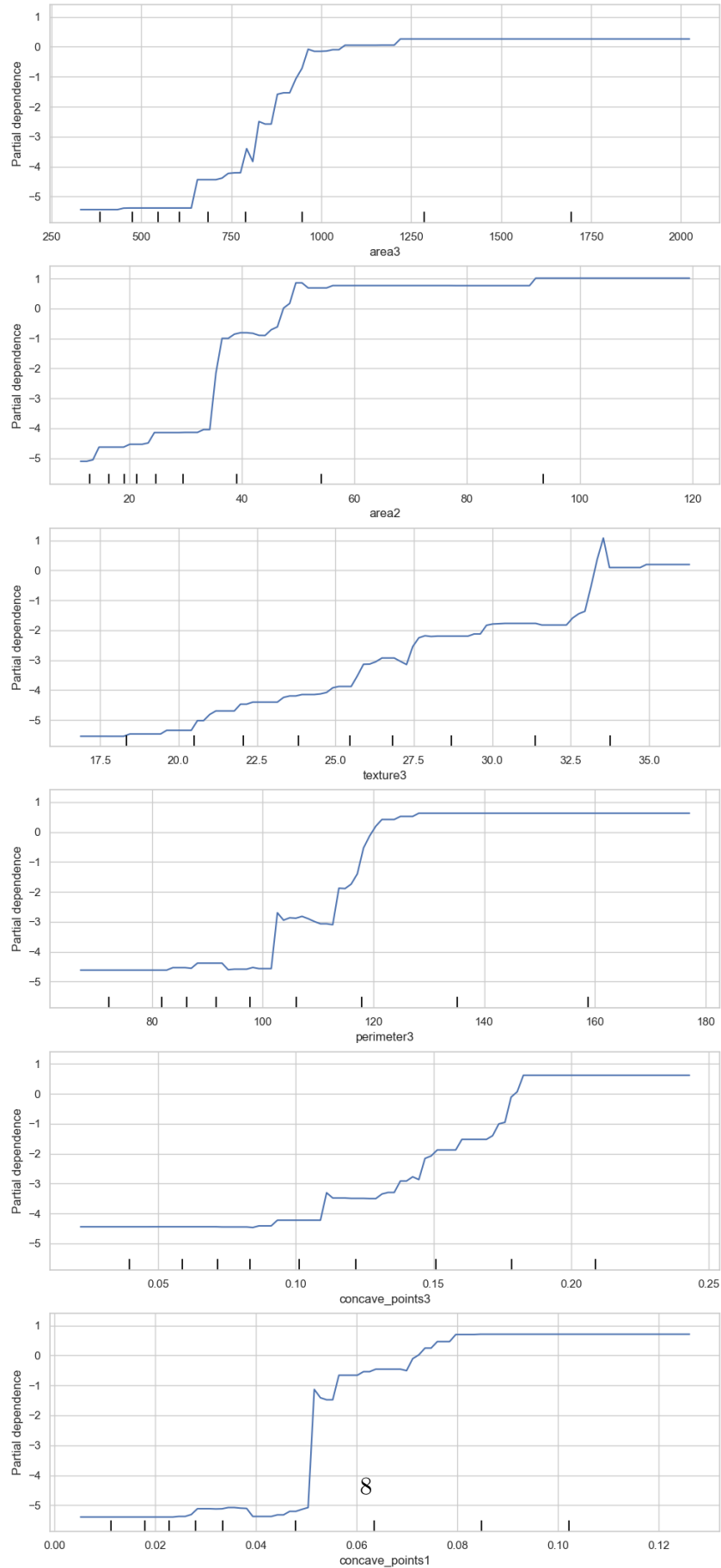
top_features = wisconsin_importance["feature"].head(6).tolist()

fig, ax = plt.subplots(figsize=(10, 22))
PartialDependenceDisplay.from_estimator(
    wisconsin_gb,
    X_w_train,
    features=top_features,
    n_cols=1,
    ax=ax,
)
plt.suptitle("Wisconsin Partial Dependence Plots for Top 6 Features", y=1.01)
plt.tight_layout()
plt.show()

```



Wisconsin Partial Dependence Plots for Top 6 Features



1.1.1 Wisconsin Takeaway

For Wisconsin, regular gradient boosting works well because the dataset is small enough that I can tune it without making the workflow too heavy.

The graphs that help the most here are the ROC curve, precision-recall curve, confusion matrix, training deviance over the boosting iterations, and the feature interpretation plots.

1.2 2. Brazil Dataset

The Brazil dataset needed a different approach because it is much larger and the target is more imbalanced.

Instead of copying the exact same setup from Wisconsin, I used a more practical workflow: sample a manageable number of rows, keep a random forest baseline, use HistGradientBoostingClassifier, and pay more attention to ROC AUC, average precision, and threshold tradeoffs.

```
[24]: brazil_sample_size = 120_000

brazil_df, brazil_target = load_brazil(sample_size=brazil_sample_size,
    ↪random_state=42, rename_cols=True)
X_b, y_b = split_X_y(brazil_df, brazil_target)

print("Brazil sample shape:", brazil_df.shape)
display(y_b.value_counts().rename("count").to_frame())
display((y_b.value_counts(normalize=True).rename("share") * 100).round(2).
    ↪to_frame())

X_b_train, X_b_test, y_b_train, y_b_test = train_test_split(
    X_b, y_b, test_size=0.25, random_state=42, stratify=y_b
)

print("Training rows:", X_b_train.shape[0])
print("Test rows:", X_b_test.shape[0])
```

Brazil sample shape: (120000, 40)

	count
deceased	
0	111602
1	8398

	share
deceased	
0	93.0
1	7.0

Training rows: 90000

Test rows: 30000

```
[25]: brazil_rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    min_samples_leaf=10,
    class_weight="balanced_subsample",
    random_state=42,
    n_jobs=-1,
)
brazil_rf.fit(X_b_train, y_b_train)

brazil_hgb_grid = {
    "learning_rate": [0.03, 0.05, 0.1],
    "max_depth": [3, 5, None],
    "max_leaf_nodes": [15, 31],
    "min_samples_leaf": [50, 200],
    "l2_regularization": [0.0, 1.0],
}

brazil_hgb_search = GridSearchCV(
    estimator=HistGradientBoostingClassifier(
        max_iter=250,
        early_stopping=True,
        validation_fraction=0.1,
        random_state=42,
    ),
    param_grid=brazil_hgb_grid,
    scoring="average_precision",
    cv=3,
    n_jobs=-1,
)
brazil_hgb_search.fit(X_b_train, y_b_train)

brazil_hgb = brazil_hgb_search.best_estimator_
print("Best Brazil HistGradientBoosting parameters:")
print(brazil_hgb_search.best_params_)
```

Best Brazil HistGradientBoosting parameters:

```
{'l2_regularization': 0.0, 'learning_rate': 0.05, 'max_depth': None,
 'max_leaf_nodes': 31, 'min_samples_leaf': 50}
```

```
[26]: brazil_results = []

for model_name, model in {
    "Random Forest": brazil_rf,
    "HistGradientBoosting": brazil_hgb,
```

```

}.items():
    y_pred = model.predict(X_b_test)
    y_score = model.predict_proba(X_b_test)[:, 1]

    brazil_results.append(
        {
            "model": model_name,
            "accuracy": accuracy_score(y_b_test, y_pred),
            "balanced_accuracy": balanced_accuracy_score(y_b_test, y_pred),
            "f1": f1_score(y_b_test, y_pred),
            "roc_auc": roc_auc_score(y_b_test, y_score),
            "average_precision": average_precision_score(y_b_test, y_score),
        }
    )

brazil_results_df = pd.DataFrame(brazil_results).
    ↪sort_values("average_precision", ascending=False)
display(brazil_results_df.round(4))

```

	model	accuracy	balanced_accuracy	f1	roc_auc \
1	HistGradientBoosting	0.9535	0.7350	0.5917	0.9584
0	Random Forest	0.8833	0.8793	0.5120	0.9508

	average_precision
1	0.6751
0	0.6392

```

[27]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for model_name, model, color in [
    ("Random Forest", brazil_rf, "#2563eb"),
    ("HistGradientBoosting", brazil_hgb, "#dc2626"),
]:
    y_score = model.predict_proba(X_b_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_b_test, y_score)
    precision, recall, _ = precision_recall_curve(y_b_test, y_score)

    axes[0].plot(fpr, tpr, label=model_name, color=color, linewidth=2)
    axes[1].plot(recall, precision, label=model_name, color=color, linewidth=2)

cm = confusion_matrix(y_b_test, brazil_hgb.predict(X_b_test))
sns.heatmap(cm, annot=True, fmt="d", cmap="Reds", cbar=False, ax=axes[2])

axes[0].plot([0, 1], [0, 1], linestyle="--", color="gray")
axes[0].set_title("Brazil ROC Curve")
axes[0].set_xlabel("False Positive Rate")
axes[0].set_ylabel("True Positive Rate")

```

```

axes[0].legend()

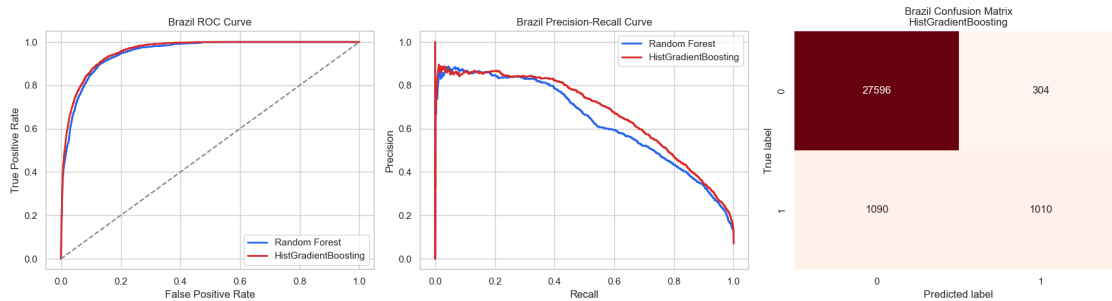
axes[1].set_title("Brazil Precision-Recall Curve")
axes[1].set_xlabel("Recall")
axes[1].set_ylabel("Precision")
axes[1].legend()

axes[2].set_title("Brazil Confusion Matrix\nHistGradientBoosting")
axes[2].set_xlabel("Predicted label")
axes[2].set_ylabel("True label")

plt.tight_layout()
plt.show()

print("Brazil classification report for HistGradientBoosting:")
print(classification_report(y_b_test, brazil_hgb.predict(X_b_test)))

```



```

Brazil classification report for HistGradientBoosting:
      precision    recall  f1-score   support

    0       0.96      0.99      0.98     27900
    1       0.77      0.48      0.59      2100

   accuracy              0.95     30000
  macro avg       0.87      0.74      0.78     30000
 weighted avg       0.95      0.95      0.95     30000

```

```

[28]: brazil_scores = brazil_hgb.predict_proba(X_b_test)[: , 1]
      thresholds = np.linspace(0.05, 0.50, 19)

      threshold_rows = []
      for threshold in thresholds:
          threshold_pred = (brazil_scores >= threshold).astype(int)
          threshold_rows.append(
              {

```

```

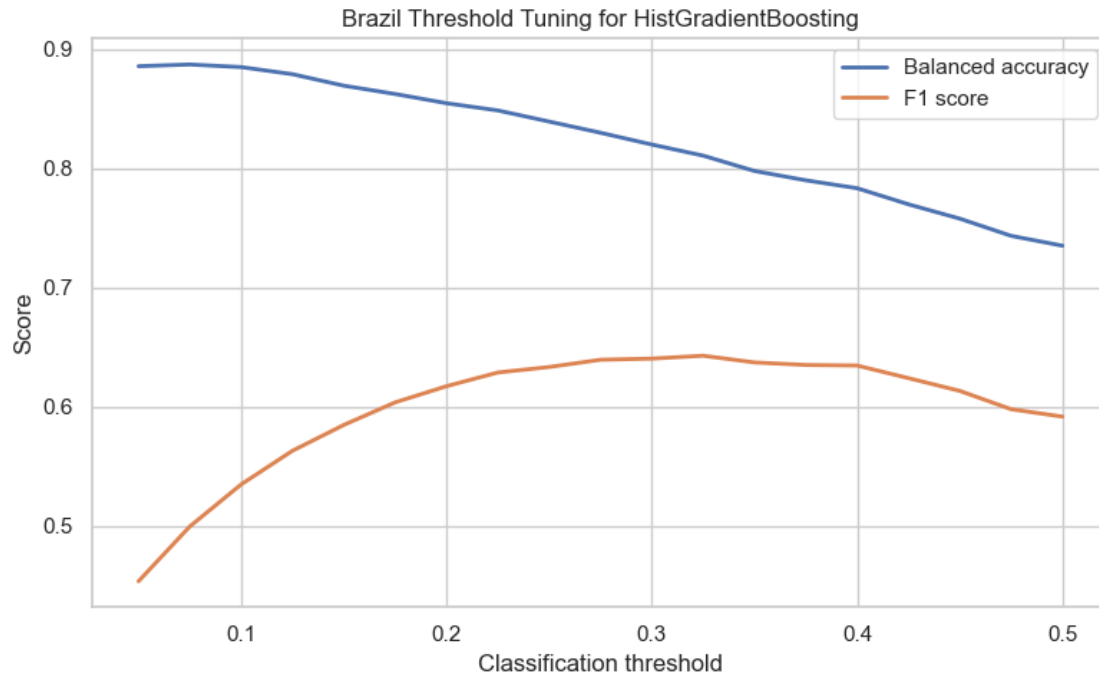
        "threshold": threshold,
        "balanced_accuracy": balanced_accuracy_score(y_b_test,
↪threshold_pred),
        "f1": f1_score(y_b_test, threshold_pred),
    }
)

threshold_df = pd.DataFrame(threshold_rows)
display(threshold_df.round(4))

plt.figure(figsize=(8, 5))
plt.plot(threshold_df["threshold"], threshold_df["balanced_accuracy"],
↪label="Balanced accuracy", linewidth=2)
plt.plot(threshold_df["threshold"], threshold_df["f1"], label="F1 score",
↪linewidth=2)
plt.title("Brazil Threshold Tuning for HistGradientBoosting")
plt.xlabel("Classification threshold")
plt.ylabel("Score")
plt.legend()
plt.tight_layout()
plt.show()

```

	threshold	balanced_accuracy	f1
0	0.050	0.8856	0.4536
1	0.075	0.8870	0.4996
2	0.100	0.8848	0.5349
3	0.125	0.8789	0.5631
4	0.150	0.8692	0.5847
5	0.175	0.8623	0.6037
6	0.200	0.8545	0.6172
7	0.225	0.8485	0.6287
8	0.250	0.8392	0.6333
9	0.275	0.8298	0.6394
10	0.300	0.8199	0.6403
11	0.325	0.8106	0.6428
12	0.350	0.7977	0.6371
13	0.375	0.7899	0.6350
14	0.400	0.7833	0.6345
15	0.425	0.7698	0.6240
16	0.450	0.7577	0.6133
17	0.475	0.7433	0.5979
18	0.500	0.7350	0.5917



```
[29]: brazil_sample_idx = X_b_test.sample(n=5000, random_state=42).index

brazil_perm = permutation_importance(
    brazil_hgb,
    X_b_test.loc[brazil_sample_idx],
    y_b_test.loc[brazil_sample_idx],
    n_repeats=10,
    random_state=42,
    scoring="average_precision",
)

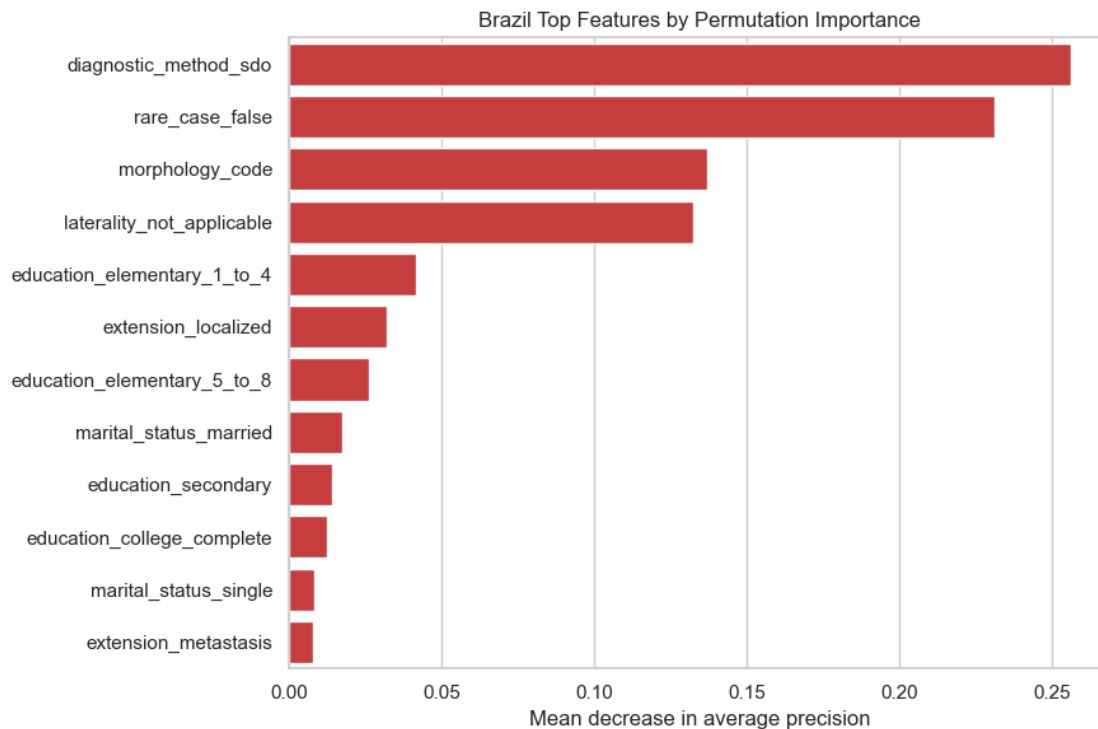
brazil_importance = (
    pd.DataFrame(
        {
            "feature": X_b.columns,
            "importance": brazil_perm.importances_mean,
        }
    )
    .sort_values("importance", ascending=False)
    .head(12)
)

plt.figure(figsize=(9, 6))
```

```

sns.barplot(data=brazil_importance, x="importance", y="feature",
            color="#dc2626")
plt.title("Brazil Top Features by Permutation Importance")
plt.xlabel("Mean decrease in average precision")
plt.ylabel("")
plt.tight_layout()
plt.show()

```



1.2.1 Brazil Takeaway

For Brazil, gradient boosting still makes sense, but I had to adjust the workflow to match the size and balance of the data.

The biggest points here are using the histogram-based version for speed, evaluating the model with class imbalance in mind, looking closely at the precision-recall curve, and checking whether a different classification threshold works better than 0.50.

```

[39]: # Report cross-validation performance for the already-selected models.
      # This refits only the winning parameter sets; it does not repeat either grid
      # search.
      from sklearn.model_selection import cross_validate

      gb_cv_scoring = {
          "roc_auc": "roc_auc",

```

```

        "average_precision": "average_precision",
    }

    wisconsin_gb_cv = cross_validate(
        wisconsin_gb,
        X_w_train,
        y_w_train,
        cv=cv,
        scoring=gb_cv_scoring,
        n_jobs=1,
    )

    brazil_gb_cv = cross_validate(
        brazil_hgb,
        X_b_train,
        y_b_train,
        cv=3,
        scoring=gb_cv_scoring,
        n_jobs=1,
    )

    gb_cv_results_df = pd.DataFrame(
        [
            {
                "dataset": "Wisconsin",
                "model": "Gradient Boosting",
                "cv_folds": 5,
                "cv_roc_auc_mean": wisconsin_gb_cv["test_roc_auc"].mean(),
                "cv_roc_auc_sd": wisconsin_gb_cv["test_roc_auc"].std(),
                "cv_pr_auc_mean": wisconsin_gb_cv["test_average_precision"].mean(),
                "cv_pr_auc_sd": wisconsin_gb_cv["test_average_precision"].std(),
            },
            {
                "dataset": "Brazil",
                "model": "HistGradientBoosting",
                "cv_folds": 3,
                "cv_roc_auc_mean": brazil_gb_cv["test_roc_auc"].mean(),
                "cv_roc_auc_sd": brazil_gb_cv["test_roc_auc"].std(),
                "cv_pr_auc_mean": brazil_gb_cv["test_average_precision"].mean(),
                "cv_pr_auc_sd": brazil_gb_cv["test_average_precision"].std(),
            },
        ]
    )

    print(gb_cv_results_df.round(4).to_string(index=False))

```

dataset	model	cv_folds	cv_roc_auc_mean	cv_roc_auc_sd
Wisconsin	Gradient Boosting	5	0.9929	0.0051

0.9909	0.0058			
Brazil	HistGradientBoosting	3	0.9577	0.0024
0.6803	0.0170			

1.3 Final Comparison Notes

What stood out to me is that the same overall idea worked on both datasets, but the implementation needed to change depending on the data.

- **Wisconsin:** I could use regular gradient boosting and spend more time on interpretation.
- **Brazil:** I needed a more efficient model setup and more careful evaluation because of the dataset size and class imbalance.

The main points I would highlight from this notebook are:

1. Gradient boosting builds trees one at a time, with each new tree trying to improve the earlier model.
2. Smaller learning rates usually mean the model needs more boosting rounds.
3. Depth, leaf size, and related settings matter because they control how easily the model can overfit.
4. For imbalanced data, accuracy by itself is not enough, so precision-recall metrics matter more.
5. Even when the method stays the same, the workflow should match the dataset you are working with.

10_week10_kmeans_silhouette

July 30, 2026

1 Week 10: K-means and silhouette score

This notebook explores unsupervised clustering on both project datasets: Wisconsin breast-cancer measurements and Brazil cancer-registry records. It is designed as the complete Week 10 Milestone 2 analysis.

The known outcome is excluded from clustering and is used afterward only to interpret whether discovered groups differ in diagnosis/outcome rate.

1.1 Milestone 2 questions

- How does k-means behave on each dataset?
- What value of k is supported by inertia and silhouette score?
- How sensitive are the clusters to scaling and feature representation?
- Are the final clusters stable across random initializations?
- Do the unsupervised groups differ in the known outcome?

These address the clarifier's depth expectations: show selection, explain behavior, test sensitivity, and interpret meaningful or unexpected results.

```
[1]: from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.metrics import adjusted_rand_score, silhouette_score
from sklearn.preprocessing import StandardScaler

from src.load_data import load_wisconsin, load_brazil

RANDOM_STATE = 42
BRAZIL_SAMPLE_SIZE = 100_000
K_VALUES = range(2, 9)
FIGURE_DIR = Path("outputs/figures/milestone2")
FIGURE_DIR.mkdir(parents=True, exist_ok=True)
```

1.2 1. Load and inspect both datasets

The project loaders are used rather than reading files directly. Brazil is sampled reproducibly because repeated k-means tuning on 1.7 million rows is unnecessarily expensive for exploratory analysis. The sample size, seed, and full-data prevalence are reported so this computational choice is transparent.

```
[2]: wisconsin_df, wisconsin_target = load_wisconsin()
brazil_full_df, brazil_target = load_brazil(sample_size=None,
↳random_state=RANDOM_STATE)
brazil_full_prevalence = brazil_full_df[brazil_target].mean()
brazil_df = brazil_full_df.sample(n=BRAZIL_SAMPLE_SIZE,
↳random_state=RANDOM_STATE)

dataset_overview = pd.DataFrame({
    "dataset": ["Wisconsin", "Brazil"],
    "rows_used": [len(wisconsin_df), len(brazil_df)],
    "predictors": [wisconsin_df.shape[1] - 1, brazil_df.shape[1] - 1],
    "missing_predictor_values": [
        wisconsin_df.drop(columns=wisconsin_target).isna().sum().sum(),
        brazil_df.drop(columns=brazil_target).isna().sum().sum(),
    ],
    "known_outcome_rate": [
        wisconsin_df[wisconsin_target].mean(),
        brazil_full_prevalence,
    ],
})
display(dataset_overview)
print(f"Brazil full dataset rows: {len(brazil_full_df):,}")
print(f"Brazil analysis sample rows: {len(brazil_df):,};
↳random_state={RANDOM_STATE}")
```

	dataset	rows_used	predictors	missing_predictor_values	\
0	Wisconsin	569	30	0	
1	Brazil	100000	39	0	

	known_outcome_rate
0	0.372583
1	0.070407

Brazil full dataset rows: 1,742,813

Brazil analysis sample rows: 100,000; random_state=42

1.3 2. Preprocessing for Euclidean clustering

K-means minimizes squared Euclidean distance. I remove the target and diagnosis-year field, median-impute predictor missingness, and standardize every predictor. Without standardization, variables measured in large units or coded with large numeric values could dominate distance calculations. The target is never passed to the clustering algorithm, avoiding target leakage.

```
[3]: def prepare_for_kmeans(df, target, scaled=True, columns=None):
    X = df.drop(columns=[target], errors="ignore").copy()
    X = X.drop(columns=["diagnosis_year"], errors="ignore")
    if columns is not None:
        X = X[columns]
    X = X.select_dtypes(include=[np.number])
    X_imputed = SimpleImputer(strategy="median").fit_transform(X)
    X_ready = StandardScaler().fit_transform(X_imputed) if scaled else X_imputed
    return X_ready, list(X.columns)

X_w, w_features = prepare_for_kmeans(wisconsin_df, wisconsin_target)
X_b, b_features = prepare_for_kmeans(brazil_df, brazil_target)
print("Wisconsin matrix:", X_w.shape)
print("Brazil matrix:", X_b.shape)
```

Wisconsin matrix: (569, 30)

Brazil matrix: (100000, 39)

1.4 3. Select k with inertia and silhouette score

Inertia must decrease as k increases, so it is an elbow diagnostic rather than a standalone selection rule. Silhouette balances cohesion and separation; higher values indicate that observations are closer to their assigned cluster than to the nearest alternative. I test k=2 through k=8 using 20 initializations per fit.

```
[4]: def tune_k(X, dataset_name):
    rows, models = [], {}
    for k in K_VALUES:
        model = KMeans(n_clusters=k, n_init=20, random_state=RANDOM_STATE)
        labels = model.fit_predict(X)
        score = silhouette_score(
            X, labels, sample_size=min(10_000, len(X)),
            random_state=RANDOM_STATE
        )
        rows.append({
            "dataset": dataset_name,
            "k": k,
            "inertia": model.inertia_,
            "silhouette": score,
        })
        models[k] = model
    results = pd.DataFrame(rows)
    best_k = int(results.loc[results["silhouette"].idxmax(), "k"])
    return results, models, best_k

wisconsin_tuning, wisconsin_models, wisconsin_best_k = tune_k(X_w, "Wisconsin")
brazil_tuning, brazil_models, brazil_best_k = tune_k(X_b, "Brazil")
tuning_results = pd.concat([wisconsin_tuning, brazil_tuning], ignore_index=True)
```

```
display(tuning_results.round(4))
print("Selected k:", {"Wisconsin": wisconsin_best_k, "Brazil": brazil_best_k})
```

	dataset	k	inertia	silhouette
0	Wisconsin	2	1.159546e+04	0.3450
1	Wisconsin	3	1.006180e+04	0.3144
2	Wisconsin	4	9.257362e+03	0.2803
3	Wisconsin	5	8.557728e+03	0.1585
4	Wisconsin	6	7.970264e+03	0.1604
5	Wisconsin	7	7.540319e+03	0.1532
6	Wisconsin	8	7.192192e+03	0.1392
7	Brazil	2	3.684692e+06	0.1932
8	Brazil	3	3.547194e+06	0.1950
9	Brazil	4	3.438016e+06	0.0634
10	Brazil	5	3.314127e+06	0.0706
11	Brazil	6	3.213726e+06	0.0691
12	Brazil	7	3.098210e+06	0.0718
13	Brazil	8	2.978843e+06	0.1035

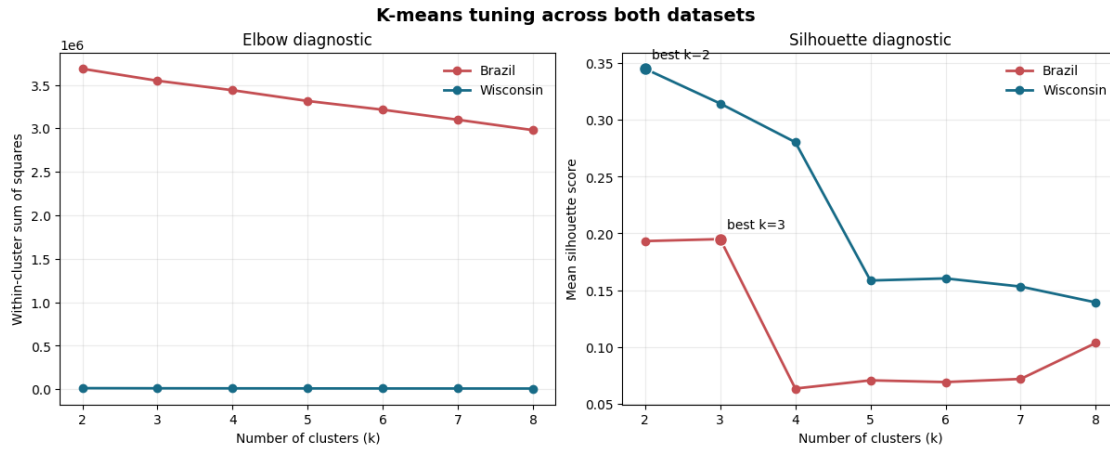
Selected k: {'Wisconsin': 2, 'Brazil': 3}

```
[5]: fig, axes = plt.subplots(1, 2, figsize=(12, 4.8), constrained_layout=True)
colors = {"Wisconsin": "#176b87", "Brazil": "#c44e52"}
for dataset, group in tuning_results.groupby("dataset"):
    axes[0].plot(group.k, group.inertia, marker="o", linewidth=2,
    ↳label=dataset, color=colors[dataset])
    axes[1].plot(group.k, group.silhouette, marker="o", linewidth=2,
    ↳label=dataset, color=colors[dataset])
    best = group.loc[group.silhouette.idxmax()]
    axes[1].scatter(best.k, best.silhouette, s=100, color=colors[dataset],
    ↳edgecolor="white", zorder=4)
    axes[1].annotate(f"best k={int(best.k)}", (best.k, best.silhouette),
    ↳xytext=(5, 8), textcoords="offset points")
axes[0].set(title="Elbow diagnostic", xlabel="Number of clusters (k)",
    ↳ylabel="Within-cluster sum of squares")
axes[1].set(title="Silhouette diagnostic", xlabel="Number of clusters (k)",
    ↳ylabel="Mean silhouette score")
for ax in axes:
    ax.set_xticks(list(K_VALUES))
    ax.grid(alpha=.25)
    ax.legend(frameon=False)
fig.suptitle("K-means tuning across both datasets", fontsize=14,
    ↳fontweight="bold")
fig.savefig(FIGURE_DIR / "week10_kmeans_tuning.png", dpi=300,
    ↳bbox_inches="tight")
plt.show()
```

```

for dataset, group in tuning_results.groupby("dataset"):
    best = group.loc[group.silhouette.idxmax()]
    print(f"{dataset}: selected k={int(best.k)}, silhouette={best.silhouette:.3f}, inertia={best.inertia:,.1f}")

```



Brazil: selected k=3, silhouette=0.195, inertia=3,547,194.4

Wisconsin: selected k=2, silhouette=0.345, inertia=11,595.5

1.5 4. Final clusters and stability

I use the silhouette-selected k for the remaining analysis. K-means can be sensitive to its random starting centroids, so I refit the selected solution with five different random seeds and compare assignments using adjusted Rand index (ARI). High ARI indicates that the grouping is reproducible despite label permutation.

```

[6]: final_w = wisconsin_models[wisconsin_best_k]
    final_b = brazil_models[brazil_best_k]
    labels_w, labels_b = final_w.labels_, final_b.labels_

def stability_table(X, k, dataset):
    reference = KMeans(n_clusters=k, n_init=20, random_state=RANDOM_STATE).
    fit_predict(X)
    rows = []
    for seed in [0, 1, 2, 3, 4]:
        candidate = KMeans(n_clusters=k, n_init=20, random_state=seed).
        fit_predict(X)
        rows.append({
            "dataset": dataset,
            "seed": seed,
            "k": k,
            "ARI_vs_seed_42": adjusted_rand_score(reference, candidate),
        })

```

```

return pd.DataFrame(rows)

stability_results = pd.concat([
    stability_table(X_w, wisconsin_best_k, "Wisconsin"),
    stability_table(X_b, brazil_best_k, "Brazil"),
], ignore_index=True)
display(stability_results.round(3))
display(stability_results.groupby("dataset")["ARI_vs_seed_42"].agg(["mean",
↪ "min", "max"]).round(3))

```

	dataset	seed	k	ARI_vs_seed_42
0	Wisconsin	0	2	1.000
1	Wisconsin	1	2	1.000
2	Wisconsin	2	2	1.000
3	Wisconsin	3	2	0.965
4	Wisconsin	4	2	1.000
5	Brazil	0	3	0.062
6	Brazil	1	3	0.022
7	Brazil	2	3	0.995
8	Brazil	3	3	0.641
9	Brazil	4	3	0.398

	mean	min	max
dataset			
Brazil	0.424	0.022	0.995
Wisconsin	0.993	0.965	1.000

The repeated-start ARI is the k-means analogue of a stability/overfitting check. `n_init=20` reduces dependence on one unlucky initialization, while repeated seeds reveal whether the selected solution is intrinsically unstable. This does not prove generalization to a new population; an additional sample or external validation would be needed.

1.6 5. PCA visualization

PCA is used only to display the high-dimensional assignments in two dimensions. The clusters were fit in the full standardized feature spaces, so visual overlap in the PCA plot does not necessarily mean the original clusters overlap in every direction.

```

[7]: pca_w = PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(X_w)
pca_b = PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(X_b)

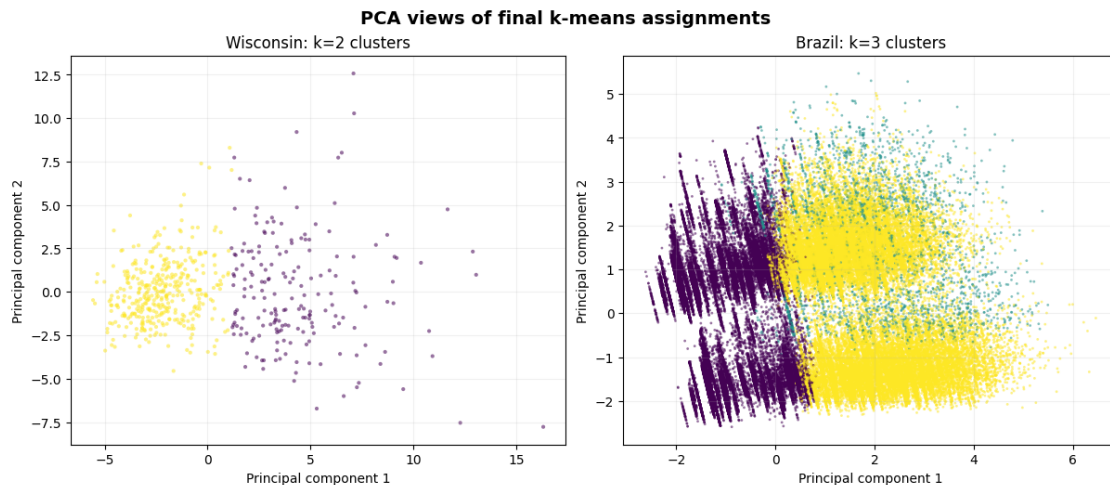
fig, axes = plt.subplots(1, 2, figsize=(12, 5.2), constrained_layout=True)
for ax, coordinates, labels, dataset, k in [
    (axes[0], pca_w, labels_w, "Wisconsin", wisconsin_best_k),
    (axes[1], pca_b, labels_b, "Brazil", brazil_best_k),
]:
    ax.scatter(
        coordinates[:, 0], coordinates[:, 1], c=labels,
        cmap="viridis", s=9 if dataset == "Wisconsin" else 4,

```

```

        alpha=.55, linewidths=0,
    )
    ax.set(title=f"{dataset}: k={k} clusters", xlabel="Principal component 1",
    ylabel="Principal component 2")
    ax.grid(alpha=.18)
fig.suptitle("PCA views of final k-means assignments", fontsize=14,
fontweight="bold")
fig.savefig(Figure_DIR / "week10_kmeans_pca_clusters.png", dpi=300,
bbox_inches="tight")
plt.show()

```



1.7 6. Interpret clusters using the known outcome

The outcome is added back only now. This table does not measure predictive accuracy: it asks whether unsupervised groups have different observed diagnosis/outcome rates. Cluster-level differences are useful for generating hypotheses, but they should not be described as causal or as validated risk strata.

```

[8]: def cluster_outcome_profile(labels, y, dataset, k):
    profile = pd.DataFrame({"cluster": labels, "outcome": np.asarray(y)})
    result = profile.groupby("cluster", as_index=False).agg(
        n=("outcome", "size"),
        outcome_rate=("outcome", "mean"),
    )
    result.insert(0, "dataset", dataset)
    result.insert(1, "k", k)
    return result

cluster_profiles = pd.concat([

```

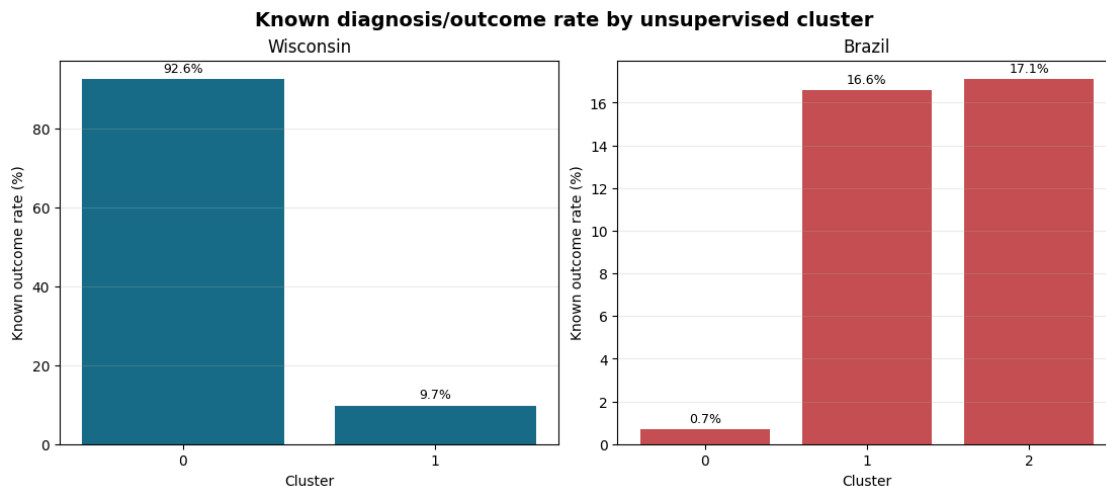
```

    cluster_outcome_profile(labels_w, wisconsin_df[wisconsin_target],
        ↪ "Wisconsin", wisconsin_best_k),
    cluster_outcome_profile(labels_b, brazil_df[brazil_target], "Brazil",
        ↪ brazil_best_k),
], ignore_index=True)
display(cluster_profiles.assign(outcome_rate=lambda d: d.outcome_rate.round(3)))

fig, axes = plt.subplots(1, 2, figsize=(11, 4.8), constrained_layout=True)
for ax, dataset, color in zip(axes, ["Wisconsin", "Brazil"], ["#176b87",
    ↪ "#c44e52"]):
    data = cluster_profiles[cluster_profiles.dataset == dataset]
    bars = ax.bar(data.cluster.astype(str), data.outcome_rate * 100,
        ↪ color=color)
    ax.bar_label(bars, fmt="%.1f%", padding=3, fontsize=9)
    ax.set(title=dataset, xlabel="Cluster", ylabel="Known outcome rate (%)")
    ax.grid(axis="y", alpha=.25)
fig.suptitle("Known diagnosis/outcome rate by unsupervised cluster",
    ↪ fontsize=14, fontweight="bold")
fig.savefig(FIGURE_DIR / "week10_kmeans_outcome_profiles.png", dpi=300,
    ↪ bbox_inches="tight")
plt.show()

```

	dataset	k	cluster	n	outcome_rate
0	Wisconsin	2	0	189	0.926
1	Wisconsin	2	1	380	0.097
2	Brazil	3	0	61328	0.007
3	Brazil	3	1	2924	0.166
4	Brazil	3	2	35748	0.171



1.8 7. Sensitivity to scaling and feature representation

Distance-based clustering depends on geometry. I compare the standardized full-feature solution with narrower representations. Wisconsin's reduced representation uses the ten mean measurement variables. Brazil's reduced representation excludes age, profession code, and morphology code, testing how much the coded variables drive the groups. Silhouette should be considered together with ARI versus the full-feature assignments.

```
[9]: def sensitivity_row(df, target, dataset, k, variant, columns=None, scaled=True,
    ↪reference=None):
    X_variant, _ = prepare_for_kmeans(df, target, scaled=scaled,
    ↪columns=columns)
    model = KMeans(n_clusters=k, n_init=20, random_state=RANDOM_STATE).
    ↪fit(X_variant)
    row = {
        "dataset": dataset,
        "variant": variant,
        "silhouette": silhouette_score(
            X_variant, model.labels_, sample_size=min(10_000, len(X_variant)),
    ↪random_state=RANDOM_STATE
        ),
    }
    if reference is not None:
        row["ARI_vs_full"] = adjusted_rand_score(reference, model.labels_)
    return row

w_reduced = [c for c in wisconsin_df.columns if c.endswith("1") and c !=
    ↪wisconsin_target]
b_reduced = [c for c in brazil_df.columns if c not in [brazil_target, "Age",
    ↪"Code.Profession", "Code.of.Morphology"]]
sensitivity_results = pd.DataFrame([
    sensitivity_row(wisconsin_df, wisconsin_target, "Wisconsin",
    ↪wisconsin_best_k, "full standardized", scaled=True, reference=labels_w),
    sensitivity_row(wisconsin_df, wisconsin_target, "Wisconsin",
    ↪wisconsin_best_k, "mean measurements standardized", columns=w_reduced,
    ↪scaled=True, reference=labels_w),
    sensitivity_row(brazil_df, brazil_target, "Brazil", brazil_best_k, "full
    ↪standardized", scaled=True, reference=labels_b),
    sensitivity_row(brazil_df, brazil_target, "Brazil", brazil_best_k,
    ↪"clinical/demographic standardized", columns=b_reduced, scaled=True,
    ↪reference=labels_b),
    sensitivity_row(wisconsin_df, wisconsin_target, "Wisconsin",
    ↪wisconsin_best_k, "full unscaled", scaled=False, reference=labels_w),
    sensitivity_row(brazil_df, brazil_target, "Brazil", brazil_best_k, "full
    ↪unscaled", scaled=False, reference=labels_b),
])
display(sensitivity_results.round(3))
```

	dataset	variant	silhouette	ARI_vs_full
0	Wisconsin	full standardized	0.345	1.000
1	Wisconsin	mean measurements standardized	0.395	0.848
2	Brazil	full standardized	0.195	1.000
3	Brazil	clinical/demographic standardized	0.195	1.000
4	Wisconsin	full unscaled	0.697	0.521
5	Brazil	full unscaled	0.796	0.050

1.9 Milestone 2 interpretation draft

K-means provides an unsupervised contrast between the datasets. Wisconsin’s continuous tumor measurements are naturally suited to standardized Euclidean geometry, while Brazil combines sparse indicators, missingness, and coded registry variables. Therefore, Brazil’s result should be interpreted more cautiously: cluster quality and stability may reflect the chosen representation as much as naturally occurring patient subgroups.

The strongest evidence for a substantive interpretation is convergence across the silhouette result, repeated-start stability, sensitivity analysis, PCA display, and outcome profiles. If these disagree, the disagreement is itself important and should be reported as a limitation. A cluster with a different outcome rate is a hypothesis-generating subgroup, not proof of a clinical mechanism or a replacement for supervised validation.

For the breadth section, summarize the setup, selected k, silhouette score, stability, and key cluster interpretation for both datasets. For the depth section, expand the geometry of scaling, walk through the k-selection evidence, explain the sensitivity results, and connect the outcome profiles to the project’s research question without claiming causation.

1.10 AI-use note for the eventual appendix

AI assistance was used to help organize the Week 10 notebook workflow, identify Milestone 2 rubric requirements, draft analysis-cell language, and suggest diagnostic comparisons. All data loading, model fitting, metrics, plots, and interpretations should be reviewed and verified by the student before submission.

11_week11_dbscan_hierarchical

July 30, 2026

1 Week 11: DBSCAN and hierarchical clustering

This notebook applies both Week 11 clustering methods to Wisconsin breast-cancer measurements and Brazil cancer-registry records. DBSCAN is the primary analysis because it can identify dense groups and label isolated observations as noise. Hierarchical agglomerative clustering is included as the companion method required by the Milestone 2 rubric.

The target is excluded from all clustering fits and is used afterward only for descriptive outcome profiles.

1.1 Milestone 2 questions

- Does density-based clustering find meaningful groups or substantial noise?
- How do DBSCAN's `eps` and `min_samples` parameters change the result?
- How does hierarchical clustering compare when the number of clusters is specified?
- Do discovered groups differ in the known diagnosis/outcome?

These address the clarifier's depth expectations: show parameter selection, explain model behavior, test sensitivity, and interpret meaningful or unexpected results.

```
[1]: from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.cluster import DBSCAN, AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

from src.load_data import load_wisconsin, load_brazil

RANDOM_STATE = 42
BRAZIL_DBSCAN_ROWS = 30_000
BRAZIL_HIERARCHICAL_ROWS = 5_000
K_VALUES = range(2, 9)
FIGURE_DIR = Path("outputs/figures/milestone2")
FIGURE_DIR.mkdir(parents=True, exist_ok=True)
```

1.2 1. Load and inspect both datasets

Wisconsin is small enough to analyze directly. DBSCAN and hierarchical clustering require neighborhood or distance calculations, so Brazil uses a documented reproducible sample. This keeps the workflow computationally practical while making the sampling choice explicit.

```
[2]: wisconsin_df, wisconsin_target = load_wisconsin()
     brazil_df, brazil_target = load_brazil(sample_size=BRAZIL_DBSCAN_ROWS,
     ↪ random_state=RANDOM_STATE)

     dataset_overview = pd.DataFrame({
         "dataset": ["Wisconsin", "Brazil"],
         "rows_used": [len(wisconsin_df), len(brazil_df)],
         "predictors": [wisconsin_df.shape[1] - 1, brazil_df.shape[1] - 1],
         "missing_predictor_values": [
             wisconsin_df.drop(columns=wisconsin_target).isna().sum().sum(),
             brazil_df.drop(columns=brazil_target).isna().sum().sum(),
         ],
         "known_outcome_rate": [
             wisconsin_df[wisconsin_target].mean(),
             brazil_df[brazil_target].mean(),
         ],
     })
     display(dataset_overview)
     print(f"Brazil sample size: {len(brazil_df):,}; random_state={RANDOM_STATE}")
```

	dataset	rows_used	predictors	missing_predictor_values	\
0	Wisconsin	569	30	0	
1	Brazil	30000	39	0	

	known_outcome_rate
0	0.372583
1	0.071400

Brazil sample size: 30,000; random_state=42

1.3 2. Preprocessing for distance-based clustering

Both methods depend on distances. I remove the target and diagnosis-year field, median-impute numeric predictor missingness, and standardize predictors so large-unit or large-code variables do not dominate. The target is never supplied to DBSCAN or hierarchical clustering, preventing target leakage.

```
[3]: def prepare_for_clustering(df, target, scaled=True, columns=None):
     X = df.drop(columns=[target], errors="ignore").copy()
     X = X.drop(columns=["diagnosis_year"], errors="ignore")
     if columns is not None:
         X = X[columns]
     X = X.select_dtypes(include=[np.number])
```

```

X = SimpleImputer(strategy="median").fit_transform(X)
return StandardScaler().fit_transform(X) if scaled else X

X_w = prepare_for_clustering(wisconsin_df, wisconsin_target)
X_b = prepare_for_clustering(brazil_df, brazil_target)
print("Wisconsin matrix:", X_w.shape)
print("Brazil matrix:", X_b.shape)

```

Wisconsin matrix: (569, 30)

Brazil matrix: (30000, 39)

1.4 3. DBSCAN parameter exploration

DBSCAN has two central parameters. `eps` is the neighborhood radius: increasing it makes more observations neighbors and usually reduces noise. `min_samples` is the minimum number of observations required for a dense core point: increasing it makes the density requirement stricter.

For each configuration I record the number of clusters, the fraction labeled noise, and the silhouette score on non-noise observations. A silhouette score is not meaningful when every point is noise or when only one non-noise cluster remains.

```

[4]: def dbscan_grid(X, dataset, eps_values, min_samples_values):
    rows, models = [], {}
    for min_samples in min_samples_values:
        for eps in eps_values:
            model = DBSCAN(eps=eps, min_samples=min_samples,
↪metric="euclidean", n_jobs=-1)
            labels = model.fit_predict(X)
            non_noise = labels != -1
            cluster_count = len(set(labels[non_noise]))
            noise_fraction = 1 - non_noise.mean()
            if cluster_count >= 2 and non_noise.sum() > 2:
                score = silhouette_score(
                    X[non_noise], labels[non_noise],
                    sample_size=min(5_000, int(non_noise.sum())),
                    random_state=RANDOM_STATE,
                )
            else:
                score = np.nan
            key = (min_samples, eps)
            rows.append({
                "dataset": dataset,
                "eps": eps,
                "min_samples": min_samples,
                "clusters_excluding_noise": cluster_count,
                "noise_fraction": noise_fraction,
                "silhouette_non_noise": score,
            })

```

```

        models[key] = model
    return pd.DataFrame(rows), models

eps_values = [0.3, 0.5, 0.7, 1.0, 1.3, 1.7, 2.2]
min_samples_values = [5, 10, 20]
dbscan_w, dbscan_w_models = dbscan_grid(X_w, "Wisconsin", eps_values,
    ↪min_samples_values)
dbscan_b, dbscan_b_models = dbscan_grid(X_b, "Brazil", eps_values,
    ↪min_samples_values)
dbscan_results = pd.concat([dbscan_w, dbscan_b], ignore_index=True)
display(dbscan_results.round(3))

```

	dataset	eps	min_samples	clusters_excluding_noise	noise_fraction \
0	Wisconsin	0.3	5	0	1.000
1	Wisconsin	0.5	5	0	1.000
2	Wisconsin	0.7	5	0	1.000
3	Wisconsin	1.0	5	0	1.000
4	Wisconsin	1.3	5	1	0.991
5	Wisconsin	1.7	5	1	0.858
6	Wisconsin	2.2	5	4	0.548
7	Wisconsin	0.3	10	0	1.000
8	Wisconsin	0.5	10	0	1.000
9	Wisconsin	0.7	10	0	1.000
10	Wisconsin	1.0	10	0	1.000
11	Wisconsin	1.3	10	0	1.000
12	Wisconsin	1.7	10	1	0.921
13	Wisconsin	2.2	10	2	0.631
14	Wisconsin	0.3	20	0	1.000
15	Wisconsin	0.5	20	0	1.000
16	Wisconsin	0.7	20	0	1.000
17	Wisconsin	1.0	20	0	1.000
18	Wisconsin	1.3	20	0	1.000
19	Wisconsin	1.7	20	0	1.000
20	Wisconsin	2.2	20	1	0.721
21	Brazil	0.3	5	324	0.403
22	Brazil	0.5	5	366	0.342
23	Brazil	0.7	5	423	0.284
24	Brazil	1.0	5	454	0.238
25	Brazil	1.3	5	520	0.201
26	Brazil	1.7	5	542	0.178
27	Brazil	2.2	5	492	0.146
28	Brazil	0.3	10	118	0.478
29	Brazil	0.5	10	136	0.418
30	Brazil	0.7	10	177	0.365
31	Brazil	1.0	10	240	0.306
32	Brazil	1.3	10	260	0.271
33	Brazil	1.7	10	273	0.246

34	Brazil	2.2	10	234	0.211
35	Brazil	0.3	20	53	0.550
36	Brazil	0.5	20	63	0.483
37	Brazil	0.7	20	77	0.438
38	Brazil	1.0	20	107	0.388
39	Brazil	1.3	20	121	0.354
40	Brazil	1.7	20	139	0.319
41	Brazil	2.2	20	124	0.270

	silhouette_non_noise
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN
5	NaN
6	0.166
7	NaN
8	NaN
9	NaN
10	NaN
11	NaN
12	NaN
13	0.416
14	NaN
15	NaN
16	NaN
17	NaN
18	NaN
19	NaN
20	NaN
21	0.390
22	0.489
23	0.490
24	0.492
25	0.475
26	0.464
27	0.391
28	0.419
29	0.521
30	0.523
31	0.500
32	0.493
33	0.479
34	0.389
35	0.401
36	0.541
37	0.518

```

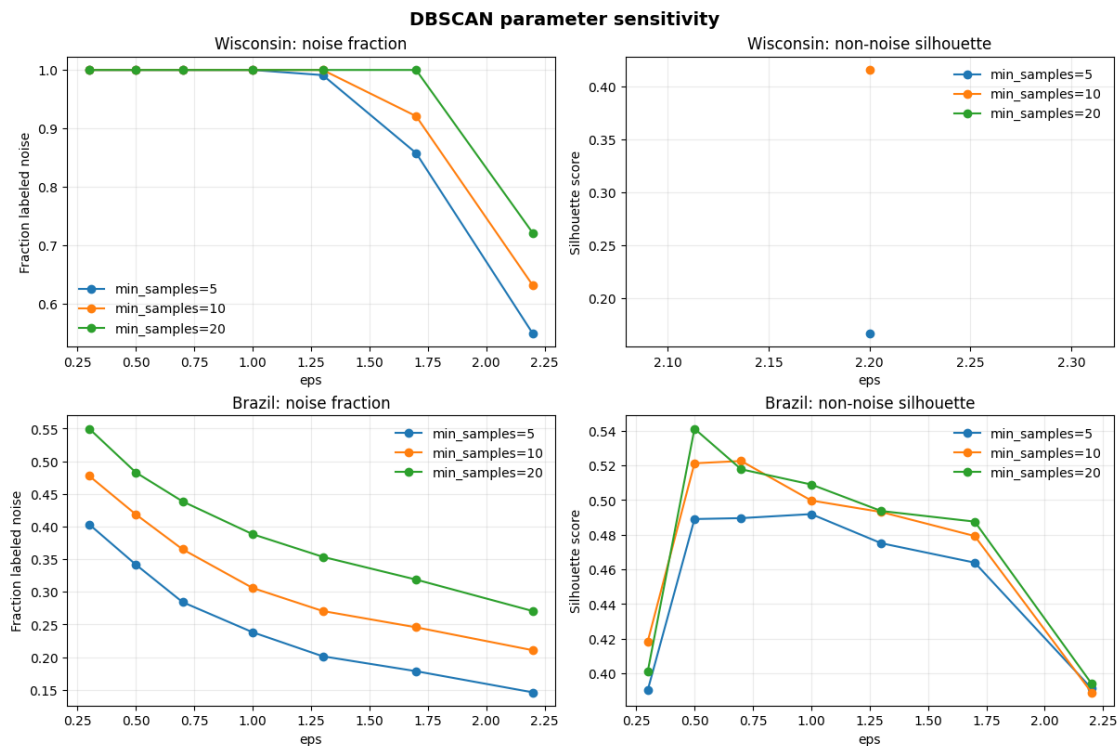
38         0.509
39         0.494
40         0.488
41         0.394

```

```

[5]: fig, axes = plt.subplots(2, 2, figsize=(12, 8), constrained_layout=True)
for row, dataset in enumerate(["Wisconsin", "Brazil"]):
    data = dbscan_results[dbscan_results.dataset == dataset]
    for min_samples in min_samples_values:
        subset = data[data.min_samples == min_samples]
        axes[row, 0].plot(subset.eps, subset.noise_fraction, marker="o",
        label=f"min_samples={min_samples}")
        axes[row, 1].plot(subset.eps, subset.silhouette_non_noise, marker="o",
        label=f"min_samples={min_samples}")
        axes[row, 0].set(title=f"{dataset}: noise fraction", xlabel="eps",
        ylabel="Fraction labeled noise")
        axes[row, 1].set(title=f"{dataset}: non-noise silhouette", xlabel="eps",
        ylabel="Silhouette score")
    for ax in axes[row]:
        ax.grid(alpha=.25)
        ax.legend(frameon=False)
fig.suptitle("DBSCAN parameter sensitivity", fontsize=14, fontweight="bold")
fig.savefig(FIGURE_DIR / "week11_dbscan_parameter_sensitivity.png", dpi=300,
        bbox_inches="tight")
plt.show()

```



1.4.1 DBSCAN selection rule

The preferred configuration is the highest non-noise silhouette among configurations with at least two clusters and no more than 80 percent noise. The noise cap prevents choosing a trivial solution that calls almost every observation an outlier. This is an exploratory rule, not a universal DBSCAN standard; the tradeoff should be explained with the project's context.

```
[6]: def select_dbscan(results):
    eligible = results[
        (results["clusters_excluding_noise"] >= 2)
        & (results["noise_fraction"] <= .80)
        & results["silhouette_non_noise"].notna()
    ]
    if eligible.empty:
        eligible = results[results["silhouette_non_noise"].notna()]
    return eligible.loc[eligible["silhouette_non_noise"].idxmax()]

best_dbscan_w = select_dbscan(dbscan_w)
best_dbscan_b = select_dbscan(dbscan_b)
display(pd.DataFrame([best_dbscan_w, best_dbscan_b]).round(3))
```

	dataset	eps	min_samples	clusters_excluding_noise	noise_fraction \
13	Wisconsin	2.2	10	2	0.631
15	Brazil	0.5	20	63	0.483

	silhouette_non_noise
13	0.416
15	0.541

1.5 4. Final DBSCAN profiles

DBSCAN is deterministic for fixed data and parameters, so its main robustness question is parameter sensitivity rather than random initialization. Noise is retained as its own descriptive category and is not silently discarded.

```
[7]: def fit_selected_dbscan(X, selected):
    model = DBSCAN(
        eps=float(selected.eps),
        min_samples=int(selected.min_samples),
        metric="euclidean",
        n_jobs=-1,
    )
    return model, model.fit_predict(X)

model_w, labels_w = fit_selected_dbscan(X_w, best_dbscan_w)
model_b, labels_b = fit_selected_dbscan(X_b, best_dbscan_b)
```

```

def dbscan_profile(labels, y, dataset, selected):
    data = pd.DataFrame({"cluster": labels, "outcome": np.asarray(y)})
    result = data.groupby("cluster", as_index=False).agg(
        n=("outcome", "size"), outcome_rate=("outcome", "mean")
    )
    result.insert(0, "dataset", dataset)
    result.insert(1, "eps", float(selected.eps))
    result.insert(2, "min_samples", int(selected.min_samples))
    return result

dbscan_profiles = pd.concat([
    dbscan_profile(labels_w, wisconsin_df[wisconsin_target], "Wisconsin",
        ↪ best_dbscan_w),
    dbscan_profile(labels_b, brazil_df[brazil_target], "Brazil", best_dbscan_b),
], ignore_index=True)
display(dbscan_profiles.assign(outcome_rate=lambda d: d.outcome_rate.round(3)))
print("Noise fraction:", {"Wisconsin": round((labels_w == -1).mean(), 3),
    "Brazil": round((labels_b == -1).mean(), 3)})

```

	dataset	eps	min_samples	cluster	n	outcome_rate
0	Wisconsin	2.2	10	-1	359	0.540
1	Wisconsin	2.2	10	0	199	0.035
2	Wisconsin	2.2	10	1	11	1.000
3	Brazil	0.5	20	-1	14480	0.137
4	Brazil	0.5	20	0	2850	0.000
..	...	...	...	...	...	...
62	Brazil	0.5	20	58	37	0.000
63	Brazil	0.5	20	59	22	0.000
64	Brazil	0.5	20	60	20	0.000
65	Brazil	0.5	20	61	28	0.000
66	Brazil	0.5	20	62	20	0.000

[67 rows x 6 columns]

Noise fraction: {'Wisconsin': np.float64(0.631), 'Brazil': np.float64(0.483)}

```

[8]: pca_w = PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(X_w)
    pca_b = PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(X_b)

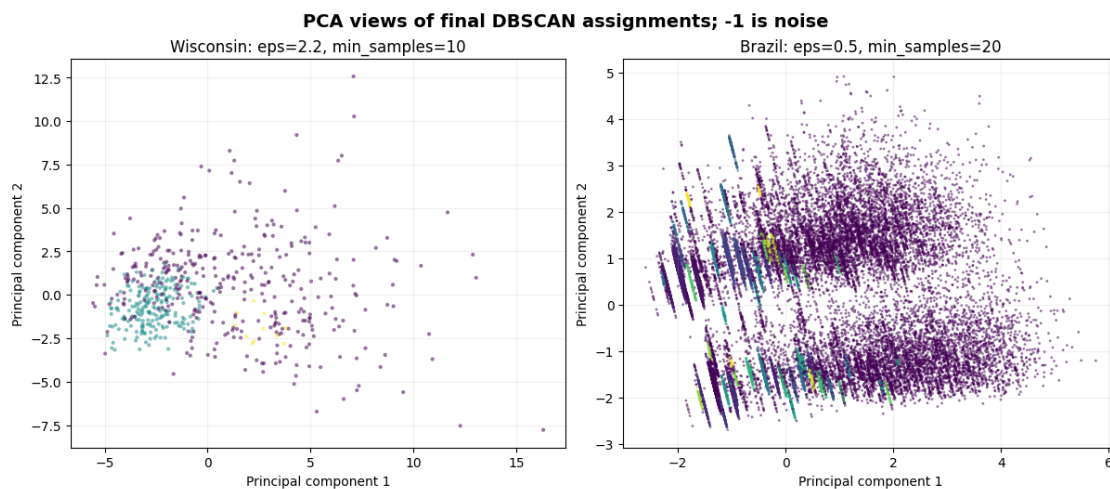
    fig, axes = plt.subplots(1, 2, figsize=(12, 5.2), constrained_layout=True)
    for ax, coordinates, labels, dataset, selected in [
        (axes[0], pca_w, labels_w, "Wisconsin", best_dbscan_w),
        (axes[1], pca_b, labels_b, "Brazil", best_dbscan_b),
    ]:
        ax.scatter(
            coordinates[:, 0], coordinates[:, 1], c=labels, cmap="viridis",
            s=9 if dataset == "Wisconsin" else 4, alpha=.55, linewidths=0,

```

```

)
ax.set(
    title=f"{dataset}: eps={selected.eps}, min_samples={int(selected.
↪min_samples)}",
    xlabel="Principal component 1", ylabel="Principal component 2",
)
ax.grid(alpha=.18)
fig.suptitle("PCA views of final DBSCAN assignments; -1 is noise", fontsize=14,
↪fontweight="bold")
fig.savefig(FIGURE_DIR / "week11_dbscan_pca_clusters.png", dpi=300,
↪bbox_inches="tight")
plt.show()

```



1.6 5. Hierarchical agglomerative clustering

Agglomerative clustering starts with each observation as its own group and repeatedly merges the closest groups. Unlike DBSCAN, it does not identify noise on its own and requires a requested number of clusters. I compare Ward and average linkage over $k=2$ through $k=8$. Ward linkage minimizes within-cluster variance; average linkage provides a sensitivity comparison.

```

[9]: def hierarchical_grid(X, dataset, row_indices=None):
    X_fit = X if row_indices is None else X[row_indices]
    rows, models = [], {}
    for linkage in ["ward", "average"]:
        for k in K_VALUES:
            model = AgglomerativeClustering(n_clusters=k, linkage=linkage)
            labels = model.fit_predict(X_fit)
            rows.append({
                "dataset": dataset,
                "linkage": linkage,

```

```

        "k": k,
        "silhouette": silhouette_score(X_fit, labels),
    })
    models[(linkage, k)] = model
    return pd.DataFrame(rows), models

b_hier_rows = np.random.default_rng(RANDOM_STATE).choice(
    len(X_b), size=min(BRAZIL_HIERARCHICAL_ROWS, len(X_b)), replace=False
)
hier_w, hier_w_models = hierarchical_grid(X_w, "Wisconsin")
hier_b, hier_b_models = hierarchical_grid(X_b, "Brazil",
    ↪row_indices=b_hier_rows)
hierarchical_results = pd.concat([hier_w, hier_b], ignore_index=True)
display(hierarchical_results.round(3))

```

	dataset	linkage	k	silhouette
0	Wisconsin	ward	2	0.339
1	Wisconsin	ward	3	0.330
2	Wisconsin	ward	4	0.298
3	Wisconsin	ward	5	0.243
4	Wisconsin	ward	6	0.120
5	Wisconsin	ward	7	0.124
6	Wisconsin	ward	8	0.096
7	Wisconsin	average	2	0.634
8	Wisconsin	average	3	0.585
9	Wisconsin	average	4	0.554
10	Wisconsin	average	5	0.455
11	Wisconsin	average	6	0.399
12	Wisconsin	average	7	0.370
13	Wisconsin	average	8	0.368
14	Brazil	ward	2	0.093
15	Brazil	ward	3	0.094
16	Brazil	ward	4	0.096
17	Brazil	ward	5	0.098
18	Brazil	ward	6	0.107
19	Brazil	ward	7	0.112
20	Brazil	ward	8	0.112
21	Brazil	average	2	0.876
22	Brazil	average	3	0.809
23	Brazil	average	4	0.771
24	Brazil	average	5	0.607
25	Brazil	average	6	0.591
26	Brazil	average	7	0.584
27	Brazil	average	8	0.584

```

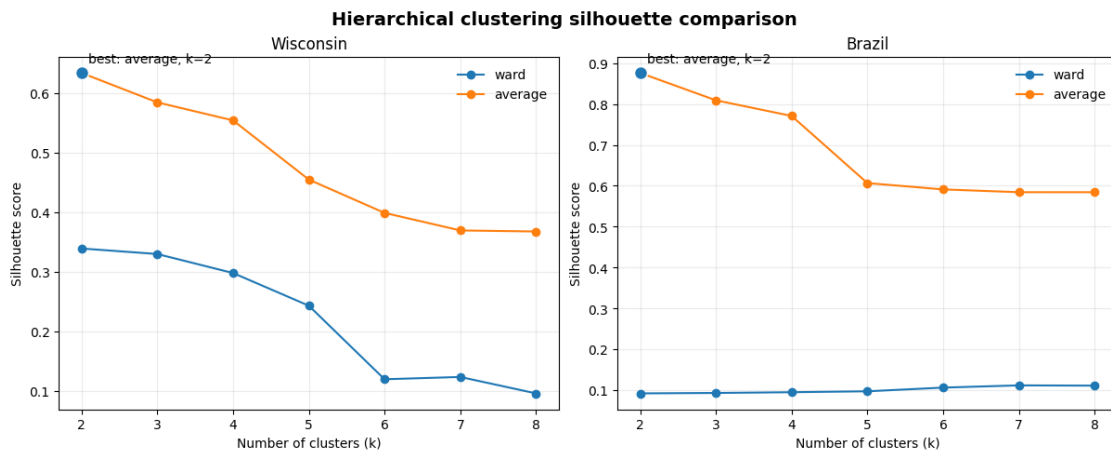
[10]: fig, axes = plt.subplots(1, 2, figsize=(12, 4.8), constrained_layout=True)
    for ax, dataset in zip(axes, ["Wisconsin", "Brazil"]):

```

```

data = hierarchical_results[hierarchical_results.dataset == dataset]
for linkage in ["ward", "average"]:
    subset = data[data.linkage == linkage]
    ax.plot(subset.k, subset.silhouette, marker="o", label=linkage)
    best = data.loc[data.silhouette.idxmax()]
    ax.scatter(best.k, best.silhouette, s=100, edgecolor="white", zorder=4)
    ax.annotate(f"best: {best.linkage}, k={int(best.k)}", (best.k, best.
    ↳silhouette), xytext=(5, 8), textcoords="offset points")
    ax.set(title=dataset, xlabel="Number of clusters (k)", ylabel="Silhouette_
    ↳score")
    ax.set_xticks(list(K_VALUES))
    ax.grid(alpha=.25)
    ax.legend(frameon=False)
fig.suptitle("Hierarchical clustering silhouette comparison", fontsize=14,
    ↳fontweight="bold")
fig.savefig(Figure_DIR / "week11_hierarchical_silhouette.png", dpi=300,
    ↳bbox_inches="tight")
plt.show()

```



1.7 6. Compare outcome profiles across methods

The outcome is added back only after clustering. These profiles are descriptive associations, not predictive metrics and not evidence that the clusters cause the outcome. For Brazil, the hierarchical result is evaluated on the documented 5,000-row sample used for pairwise clustering.

```

[11]: best_hier_w = hier_w.loc[hier_w.silhouette.idxmax()]
best_hier_b = hier_b.loc[hier_b.silhouette.idxmax()]
hier_labels_w = hier_w_models[(best_hier_w.linkage, int(best_hier_w.k))].labels_
hier_labels_b = hier_b_models[(best_hier_b.linkage, int(best_hier_b.k))].labels_

def hierarchical_profile(labels, y, dataset, selected):

```

```

data = pd.DataFrame({"cluster": labels, "outcome": np.asarray(y)})
result = data.groupby("cluster", as_index=False).agg(
    n=("outcome", "size"), outcome_rate=("outcome", "mean")
)
result.insert(0, "dataset", dataset)
result.insert(1, "linkage", selected.linkage)
result.insert(2, "k", int(selected.k))
return result

hier_profiles = pd.concat([
    hierarchical_profile(hier_labels_w, wisconsin_df.iloc[:
    ↪len(hier_labels_w)][wisconsin_target], "Wisconsin", best_hier_w),
    hierarchical_profile(hier_labels_b, brazil_df.
    ↪iloc[b_hier_rows][brazil_target], "Brazil", best_hier_b),
], ignore_index=True)
display(hier_profiles.assign(outcome_rate=lambda d: d.outcome_rate.round(3)))

```

	dataset	linkage	k	cluster	n	outcome_rate
0	Wisconsin	average	2	0	566	0.369
1	Wisconsin	average	2	1	3	1.000
2	Brazil	average	2	0	4998	0.070
3	Brazil	average	2	1	2	0.000

```

[12]: comparison = pd.DataFrame([
    {
        "dataset": "Wisconsin", "method": "DBSCAN",
        "clusters_or_noise": len(set(labels_w)) - (-1 in labels_w),
        "noise_fraction": (labels_w == -1).mean(),
        "best_internal_score": best_dbscan_w.silhouette_non_noise,
    },
    {
        "dataset": "Brazil", "method": "DBSCAN",
        "clusters_or_noise": len(set(labels_b)) - (-1 in labels_b),
        "noise_fraction": (labels_b == -1).mean(),
        "best_internal_score": best_dbscan_b.silhouette_non_noise,
    },
    {
        "dataset": "Wisconsin", "method": "Hierarchical",
        "clusters_or_noise": int(best_hier_w.k), "noise_fraction": 0,
        "best_internal_score": best_hier_w.silhouette,
    },
    {
        "dataset": "Brazil", "method": "Hierarchical",
        "clusters_or_noise": int(best_hier_b.k), "noise_fraction": 0,
        "best_internal_score": best_hier_b.silhouette,
    },
])

```

```
display(comparison.round(3))
```

	dataset	method	clusters_or_noise	noise_fraction	\
0	Wisconsin	DBSCAN	2	0.631	
1	Brazil	DBSCAN	63	0.483	
2	Wisconsin	Hierarchical	2	0.000	
3	Brazil	Hierarchical	2	0.000	

	best_internal_score
0	0.416
1	0.541
2	0.634
3	0.876

1.8 Milestone 2 interpretation draft

DBSCAN and hierarchical clustering answer different questions. DBSCAN asks whether observations form sufficiently dense regions under a chosen distance scale and allows isolated observations to remain noise. Hierarchical clustering always produces a partition at the requested k , so it is useful when a complete segmentation is desired but can hide the fact that the data may not have naturally separated groups.

The strongest evidence is the combination of parameter sensitivity, silhouette score, noise fraction, PCA display, and outcome profiles. A high silhouette with extreme noise is not automatically a good solution, and a low silhouette is evidence that the chosen representation does not support well-separated groups. Brazil requires extra caution because the registry data are heterogeneous, partly sparse, and analyzed on a sample for computational reasons.

For Milestone 2 breadth, report both DBSCAN and hierarchical results for both datasets, including parameter choices, internal metrics, noise handling, and limitations. For depth, DBSCAN is the stronger candidate: explain ϵ s and min_samples geometrically, show how the noise fraction changes, and discuss whether the selected clusters are meaningful or artifacts of scaling and representation. Outcome differences should be framed as hypothesis-generating associations rather than causal or validated clinical strata.

1.9 AI-use note for the eventual appendix

AI assistance was used to help organize the Week 11 notebook workflow, identify Milestone 2 rubric requirements, draft analysis-cell language, and suggest diagnostic comparisons. All data loading, model fitting, metrics, plots, and interpretations should be reviewed and verified by the student before submission.